

DATA 260 – HW 1

7- Community Sports League Fixtures

SJSU ID : 020053319 | SRI PRAJEET VILLYAL

Hardware: MacBook Pro, Apple M2, 8 GB RAM, macOS

Local model used: qwen2.5:3b (run locally through Ollama)

GitHub repository: <https://github.com/villyalsriprajeet-cmd/data260-3319>

Tagged commit hash: e7206b3df5b4180d931654becd0674a6d1dabaf0

Section 0 - Personal Configuration and Domain

Value	Definition
SID4	3319
PORT_BASE	8619
PREFIX	s3319
SEED	3319
VERIFY_SEED	263319
DOMAIN_ID	7

Part 1 : HTML

```
index.html U X
index.html > html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>HW1-SriPrajeet</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10 <h1>Community Sports League Fixtures</h1>
11 <form id="fixtureForm">
12   <label for="fixtureTitle">Fixture Title</label>
13   <input type="text" id="fixtureTitle" placeholder="e.g. real madrid vs barcelona – Matchweek 5" required autofocus>
14   <label for="venue">Venue</label>
15   <input type="text" id="venue" placeholder="e.g. Bernabeu Stadium" required>
16   <label for="email">Your Email</label>
17   <input type="email" id="email" placeholder="e.g. coach@example.com" required>
18   <label for="details">Match Details</label>
19   <textarea id="details" rows="4" placeholder="Describe the fixture – teams, matchweek, kickoff time" required></textarea>
20   <label for="status">Status</label>
21   <select id="status" required>
22     <option value="" disabled selected>Select status</option>
23     <option value="Scheduled">Scheduled</option>
24     <option value="Live">Live</option>
25     <option value="Completed">Completed</option>
26     <option value="Postponed">Postponed</option>
27   </select>
28   <div class="terms">
29     <input type="checkbox" id="terms">
30     <label for="terms">I agree to the terms and conditions.</label>
31   </div>
32   <button type="submit">Submit Fixture</button>
33 </form>
34 <script src="script.js"></script>
35 </body>
36 </html>
```

Created an HTML page titled HW1-Sri also added a heading tag “ <h1>Community Sports League Fixtures</h1> ” using the header tag that renders the biggest font size.

Created DOMAIN_SCHEMA.md defining all the entity fields and category values.

```
reports > hw01 > DOMAIN_SCHEMA.md > # Domain Schema > ## Dropdown options
1  # Domain Schema
2
3  Domain 7 - Community sports league fixtures.
4
5  A fixture is just a scheduled match between two teams. Each one has a title,
6  the venue it's played at, the email of whoever submitted it, some details about
7  the match, and a status showing where it's at.
8
9  ## Fields on the form
10
11 - fixtureTitle - text, required. Main field, like "Real Madrid vs Barcelona - Matchweek 5"
12 - venue - text, required. Where it's played
13 - email - email, required. Submitter's email
14 - details - textarea, required. Match description, has to be more than 25 characters
15 - status - dropdown, required. Scheduled / Live / Completed / Postponed
16 - terms - checkbox, required. Has to be ticked before submitting
17
18 ## Dropdown options
19
20 Scheduled, Live, Completed, Postponed
21
22 ## Extra field
23
24 submissionDate - not on the form. The JavaScript adds this automatically with the
25 current date/time when the form is submitted.
```

Form fields :

- 1) Fixture Title is the primary field - a required text input with a placeholder example, and I added the autofocus attribute so the cursor is already sitting in this box the moment the page loads.
- 2) Venue is the secondary field - a required text input with its own placeholder.
- 3) Your Email is a required email input. Using type="email" means the browser itself checks that what's typed looks like a real email address.
- 4) Match Details is the required textarea - this is the longer description field, and it's the one the javascript later checks for length.
- 5) Status is a dropdown with exactly four options : Scheduled, Live, Completed, and Postponed.
- 6) A checkbox with the exact label "I agree to the terms and conditions."

7) A "Submit Fixture" button, with wording that fits the sports domain.

The screenshot shows a web browser window with a form titled "Community Sports League Fixtures". The form contains the following fields:

- Fixture Title**: A text input field with a placeholder "e.g. real madrid vs barcelona - Matchweek 5".
- Venue**: A text input field with a placeholder "e.g. Bernabeu Stadium".
- Your Email**: A text input field with a placeholder "e.g. coach@example.com".
- Match Details**: A text input field with a placeholder "Describe the fixture - teams, matchweek, kickoff time".
- Status**: A dropdown menu with the text "Select status".
- Submit Fixture**: A button at the bottom of the form.

The close-up shows the "Status" dropdown menu. The options are:

- ✓ Select status
- Scheduled
- Live
- Completed
- Postponed

Added a script tag to link my JavaScript code for Part II at the end of my HTML file.

```
34 | <script src="script.js"></script>
35 | </body>
36 | </html>
```

II. Javascript

Created a javascript file called script.js

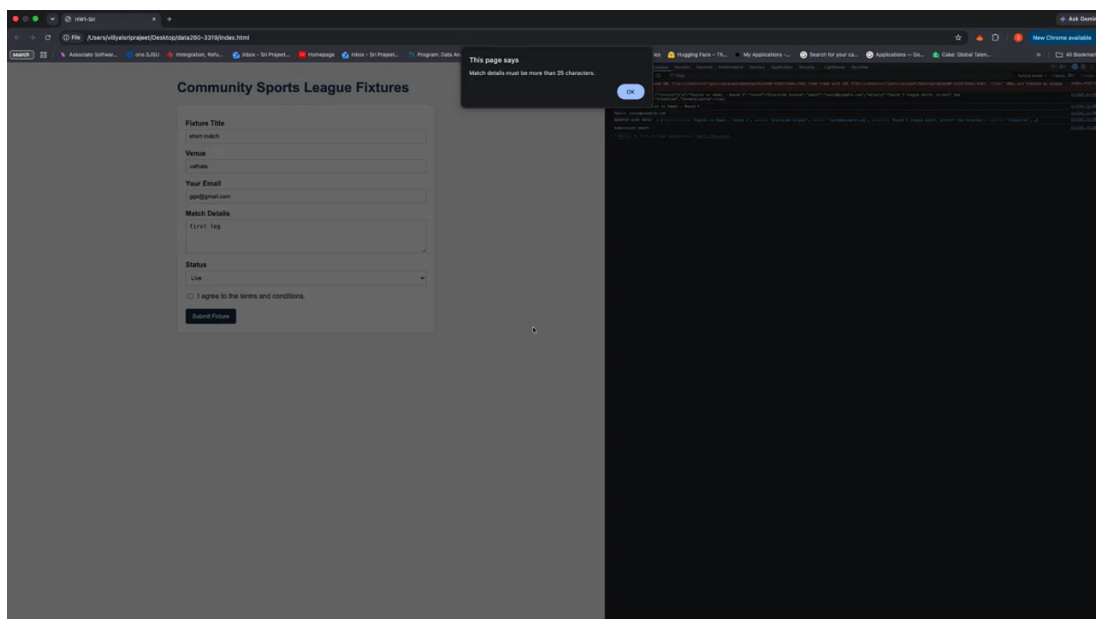
```
JS script.js > ...
37     details: document.getElementById("details").value,
38     status: document.getElementById("status").value,
39     termsAccepted: document.getElementById("terms").checked
40 };
41
42     const jsonString = JSON.stringify(fixture);
43     console.log("Form data as JSON:", jsonString);
44
45     const parsed = JSON.parse(jsonString);
46     const { fixtureTitle, email } = parsed;
47     console.log("Fixture Title:", fixtureTitle);
48     console.log("Email:", email);
49
50     const updated = { ...parsed, submissionDate: new Date().toISOString() };
51     console.log("Updated with date:", updated);
52
53     const count = submissionCounter();
54     console.log("Submission count:", count);
55
56     form.reset();
57     alert("Fixture submitted! Check the console.");
58 }));
```

It runs when the form is submitted.

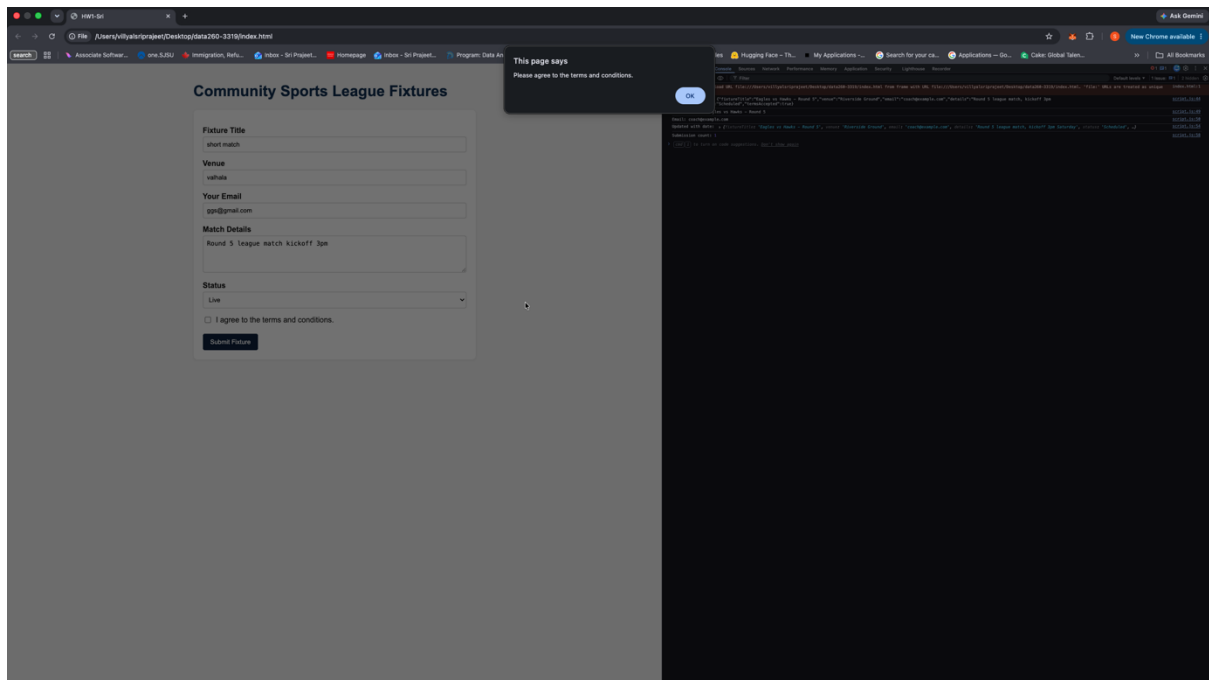
1. Validation with an arrow function

I wrote the validation as an arrow function called validateForm. It checks two things:

a) That the Match Details field has more than 25 characters. If it doesn't, it shows an alert saying "Match details must be more than 25 characters." and stops the submission.



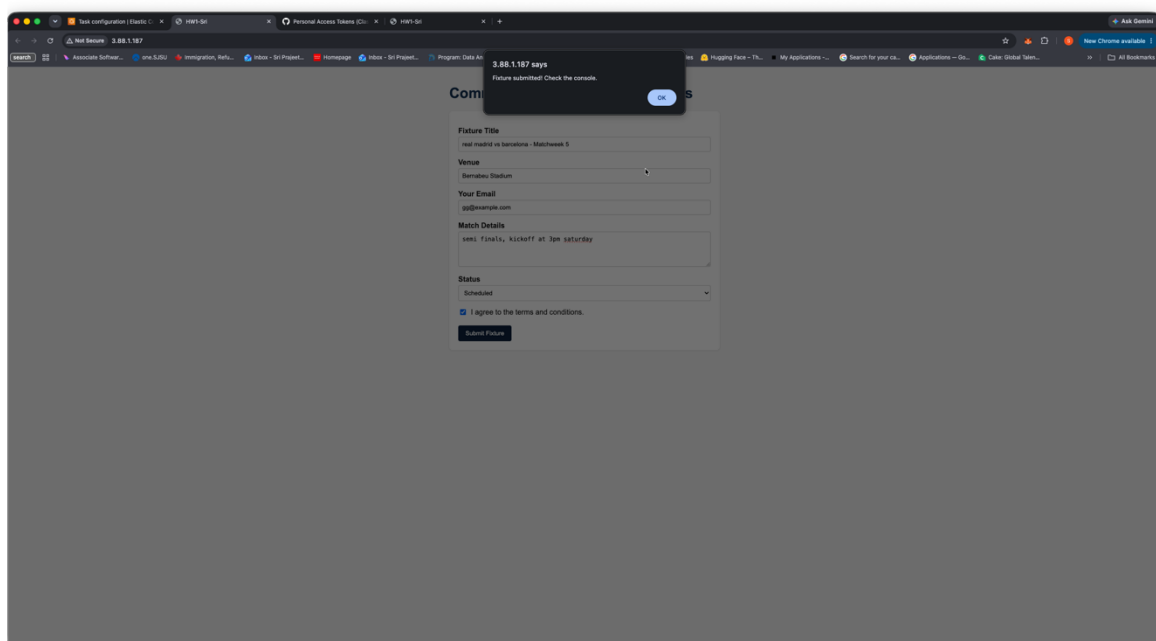
b) That the terms and conditions checkbox is ticked. If it isn't, it shows an alert saying "Please agree to the terms and conditions." and stops.



If either check fails the form doesn't submit, so nothing gets logged. Only when both pass does the rest of the code run.

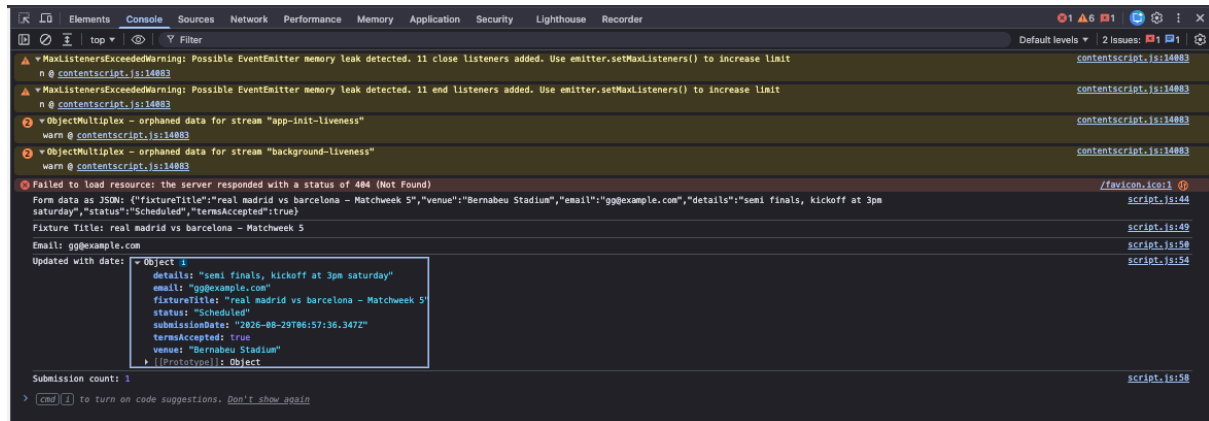
2. Converting the form data to a JSON string

Once the form is valid, I collect all the fields into one object and use `JSON.stringify` to turn that object into a JSON string, then log it to the console.



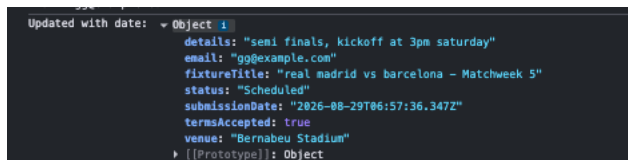
3. Object destructuring

I parse the JSON string back into an object with `JSON.parse`, then use object destructuring to pull the fixture title and the email straight out of it in one line, and log both.



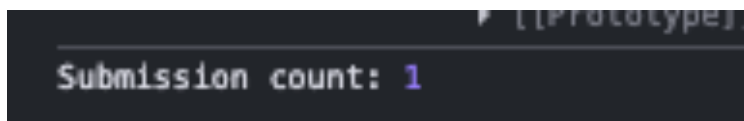
4. Spread operator to add a field

Using the spread operator, I make a copy of the parsed object and add a new field, `submissionDate`, set to the current date and time. Then I log the updated object. The spread operator copies everything from the original object and lets me tack on the new field without changing the original.



5. Closure for the submission count

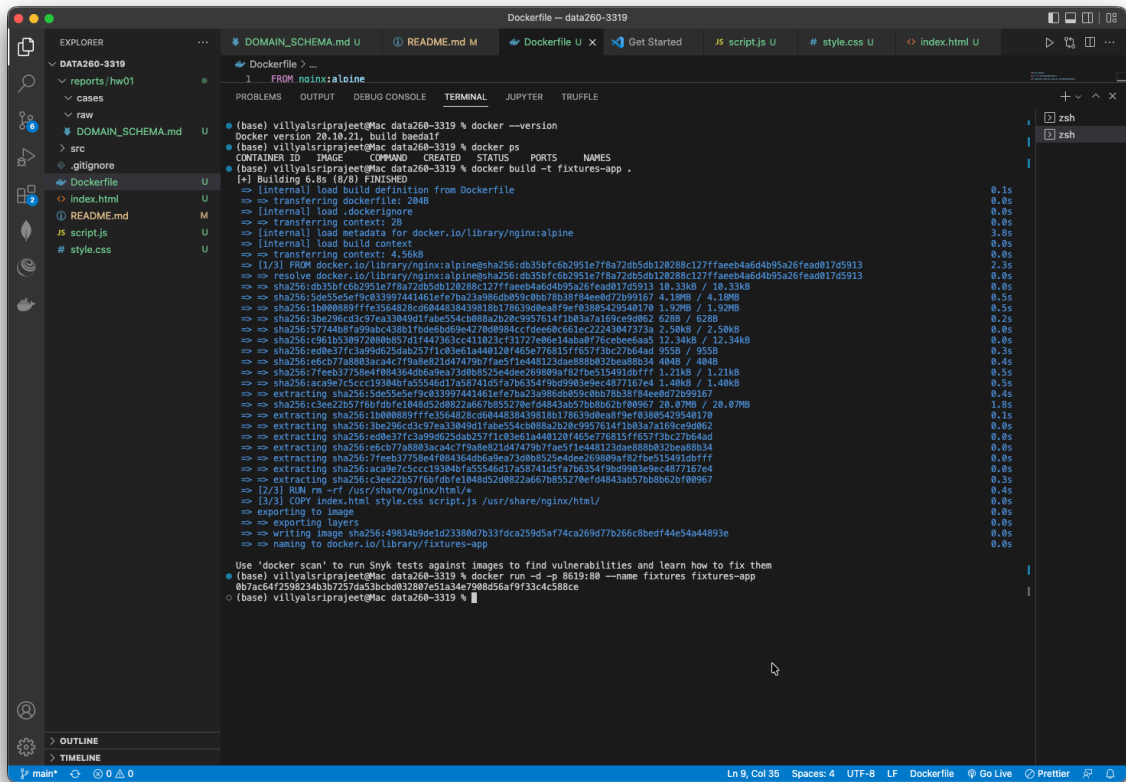
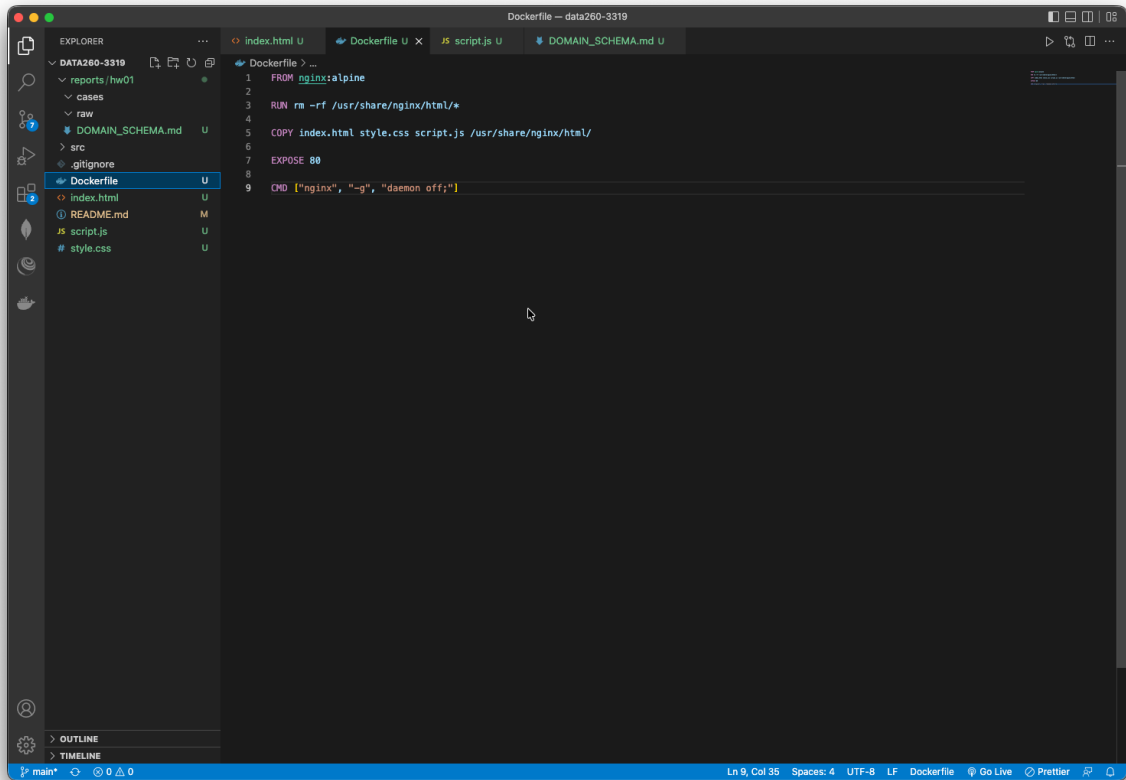
I made a closure called `submissionCounter` to keep track of how many times the form has been submitted successfully.



DEPLOYMENT:

Docker:

Created a docker image of the application and built it and ran the application using docker in my local successfully.



App running on localhost: 8619

localhost:8619
Associate Software...
one.SJSU
Immigration, Refu...
Inbox - Sri Prajeet...
Homepage
Inbox - Sri Prajeet...
Program: Data Ana...
Credia
Career Opportuniti...
Careers At Compa...
Files
Hugging Face - Th...
My

Community Sports League Fixtures

Fixture Title
e.g. real madrid vs barcelona - Matchweek 5

Venue
e.g. Bernabeu Stadium

Your Email
e.g. coach@example.com

Match Details
Describe the fixture - teams, matchweek, kickoff time

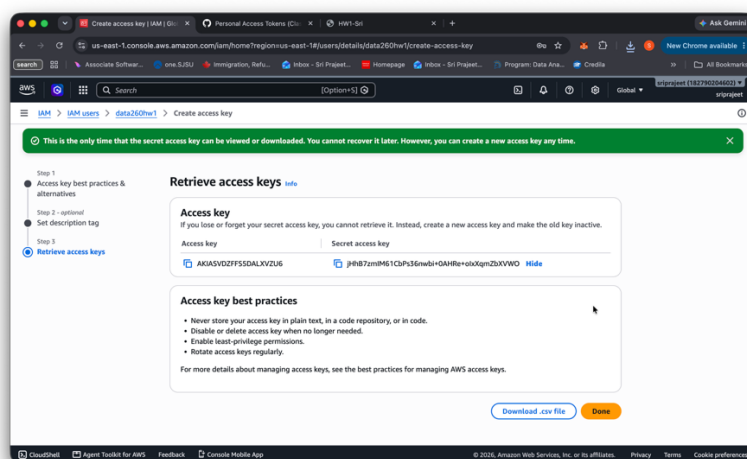
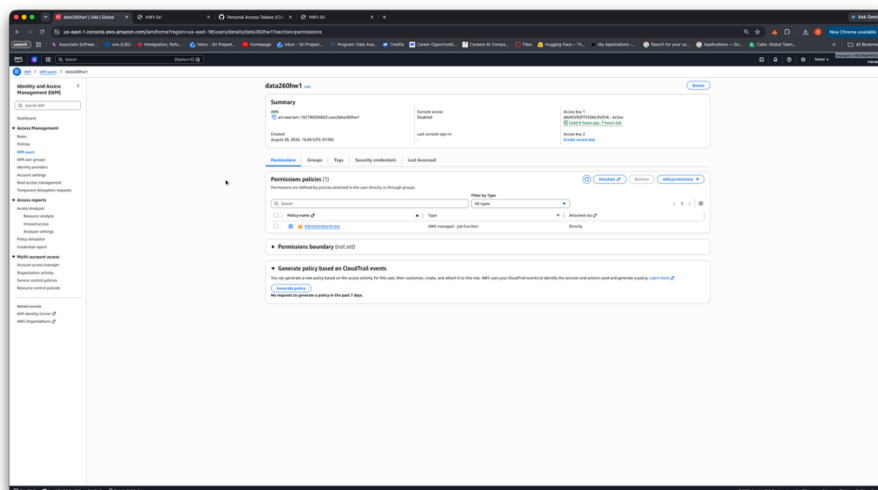
Status
Select status

☐ I agree to the terms and conditions.

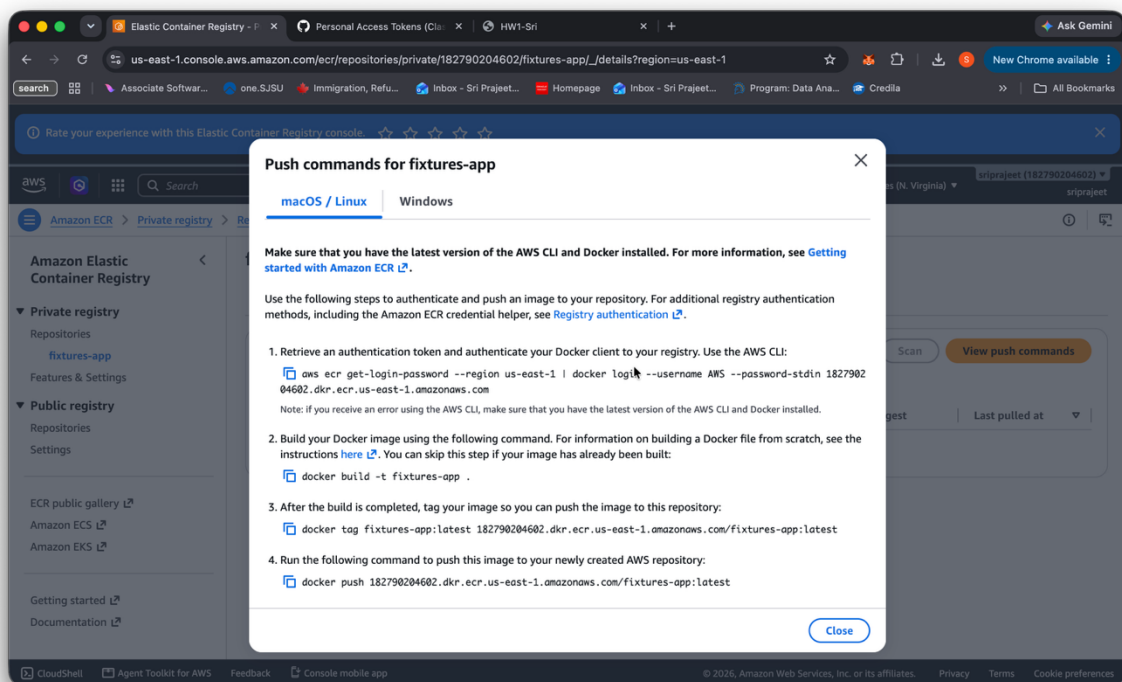
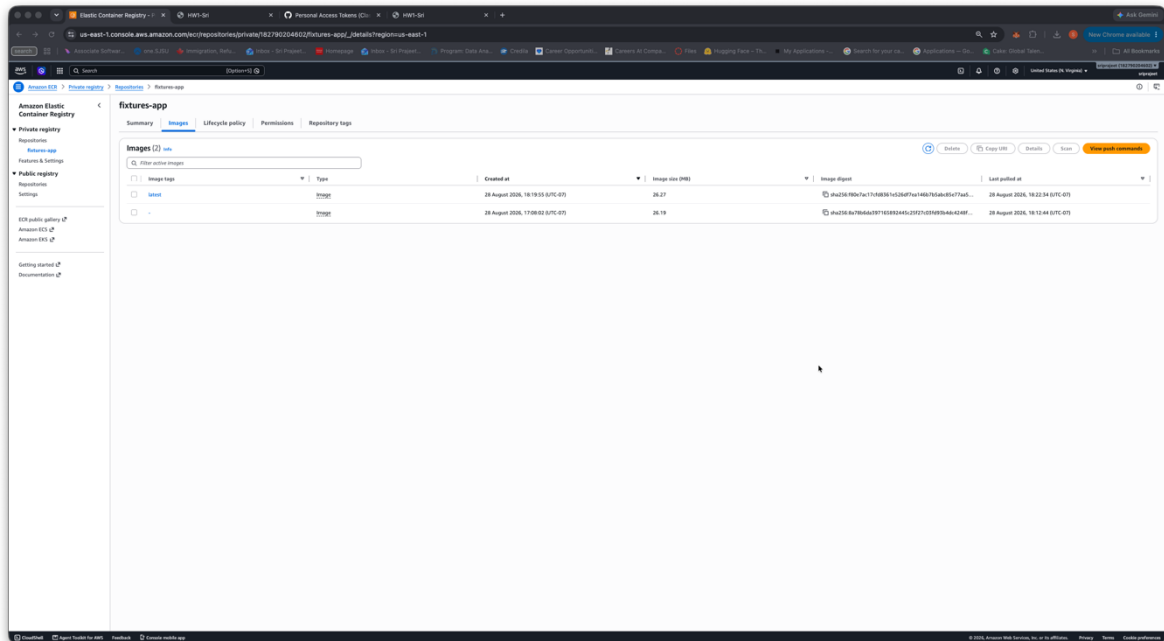
Submit Fixture

AWS ECS:

Created IAM user with admin access and created access keys

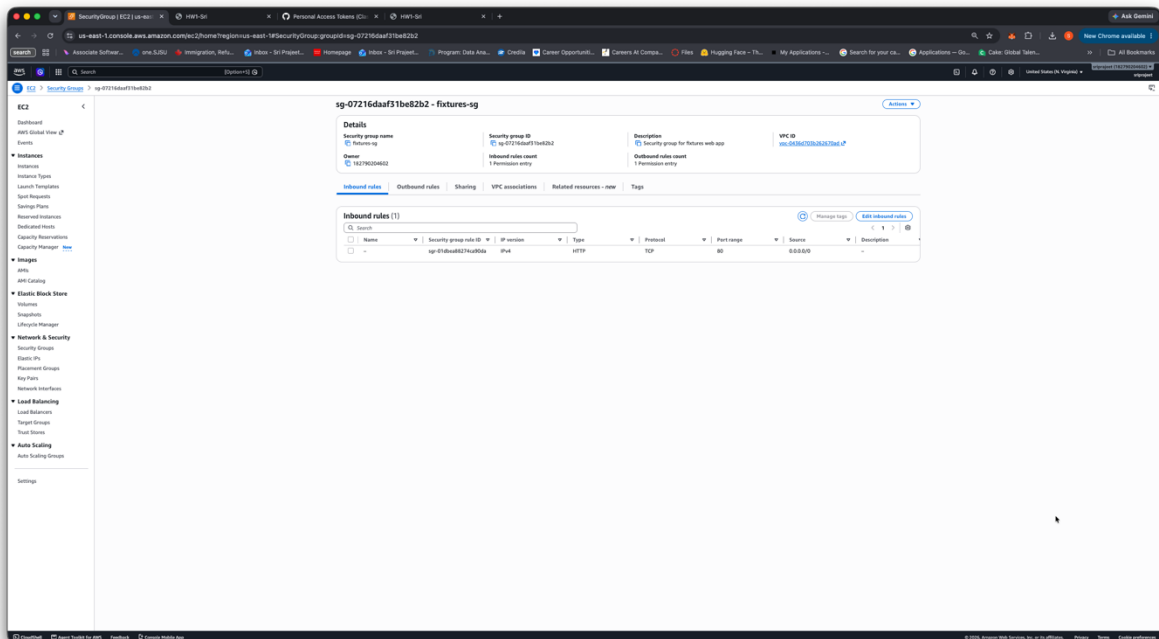


Created a private ECR repository called fixtures-app this is where AWS stores the Docker image so ECS can run it.

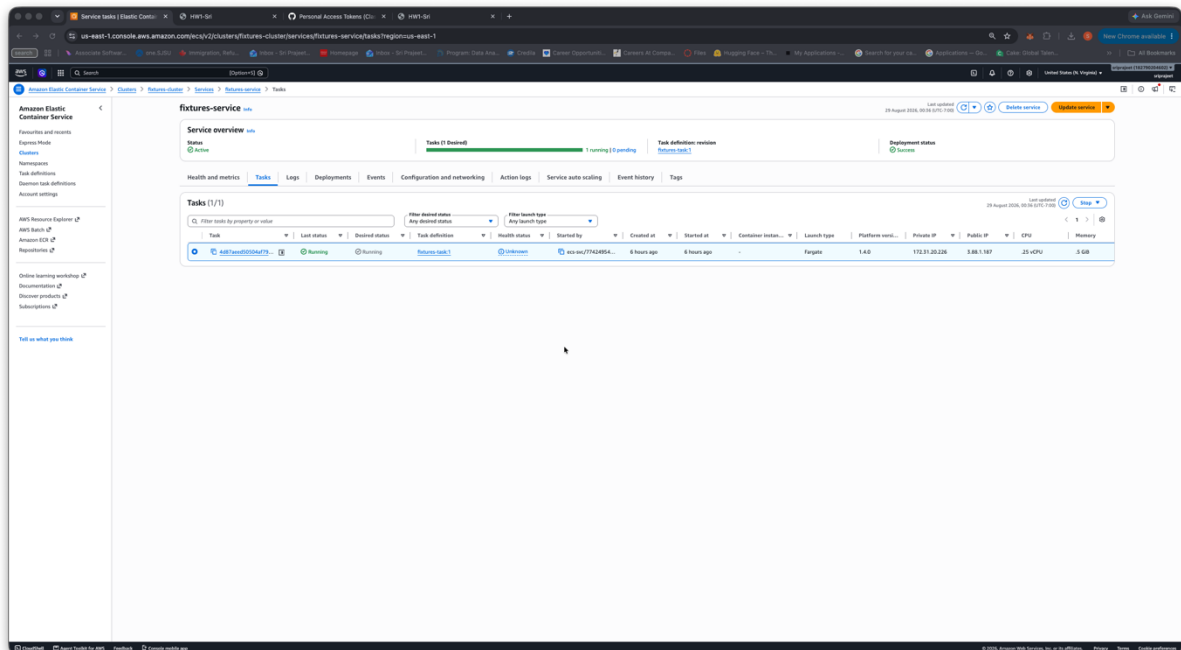
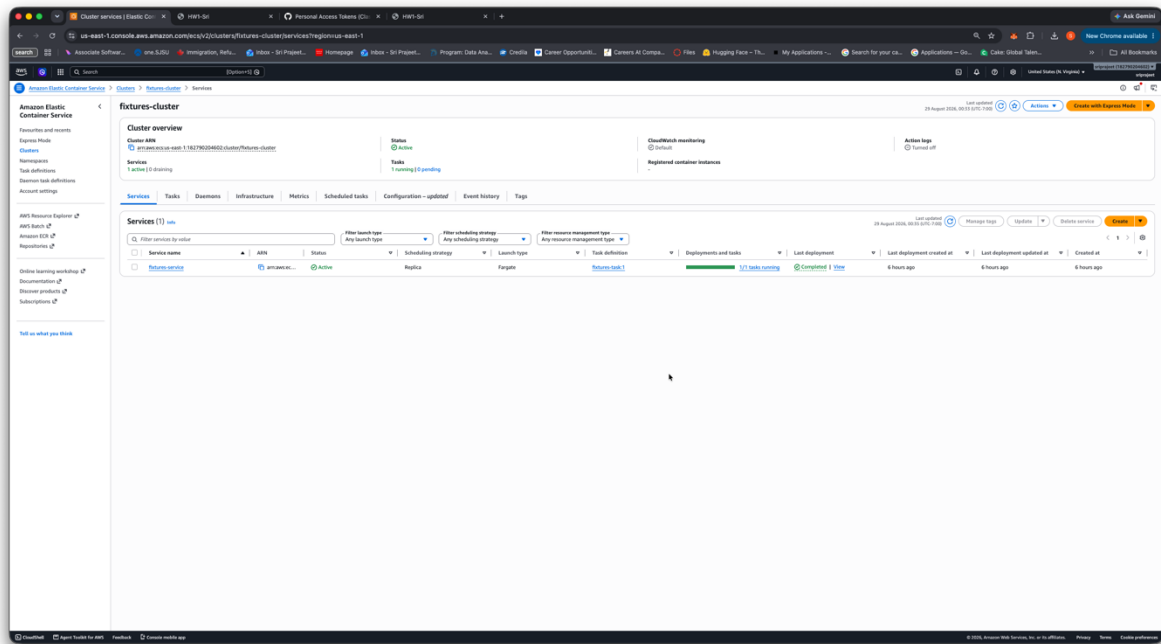


```
index.html - data260-3319
(base) villyaliriprajeet@Villyalys-MacBook-Pro data260-3319 % aws ecr get-login-password --region us-east-1 | docker login --username AWS --pass
wordst5t0ia 18279834482.dkr.ecr.us-east-1.amazonaws.com
Login Succeeded
(base) villyaliriprajeet@Villyalys-MacBook-Pro data260-3319 % docker build -t fixtures-app .
[+] Building 1.5s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 378
=> [internal] load .dockerignore
=> [internal] load metadata for docker.io/library/nginx:alpine
=> [internal] load build context
=> => transferring context: 2388
=> [1/3] FROM docker.io/library/nginx:alpine@sha256:db39fc6b2951c7f8a72d5db128288c127ffaeb4a6d4b95a26fead017d5913
=> CACHED [2/3] RUN rm -rf /usr/share/nginx/html/*
=> CACHED [3/3] COPY index.html style.css script.js /usr/share/nginx/html/
=> exporting image
=> writing image sha256:49834b9d1d23388d7b33fca259d5af74ca269d77b266cbedf44e54a44893e
=> naming to docker.io/library/fixtures-app
Use 'docker scan' to run Snyk tests against images to find vulnerabilities and learn how to fix them
(base) villyaliriprajeet@Villyalys-MacBook-Pro data260-3319 % docker tag fixtures-app:latest 18279834482.dkr.ecr.us-east-1.amazonaws.com/fixtures
-app:latest
The push refers to repository 18279834482.dkr.ecr.us-east-1.amazonaws.com/fixtures-app
44d0b81996a: Pushed
02133782a5f2: Pushed
5b2978fbc127: Pushed
7c739c0bsecr: Pushed
1601ad753516: Pushed
f79888d1be3d: Pushed
4f486d3266ae: Pushed
ae1d39f6f6b6: Pushed
8aa765947678: Pushed
b294dc92ac5f: Pushed
latest: digest: sha256:8a78b6da397165892445c25f27c83f493b4dc4248f9a273e547ad685154d3c size: 2404
(base) villyaliriprajeet@Villyalys-MacBook-Pro data260-3319 %
```

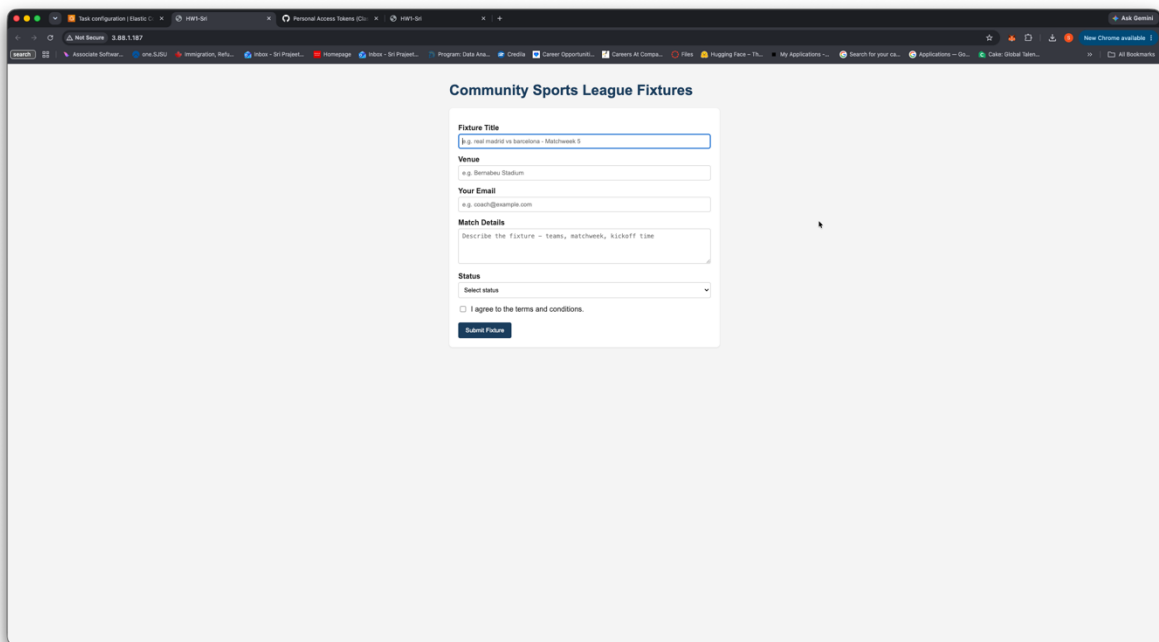
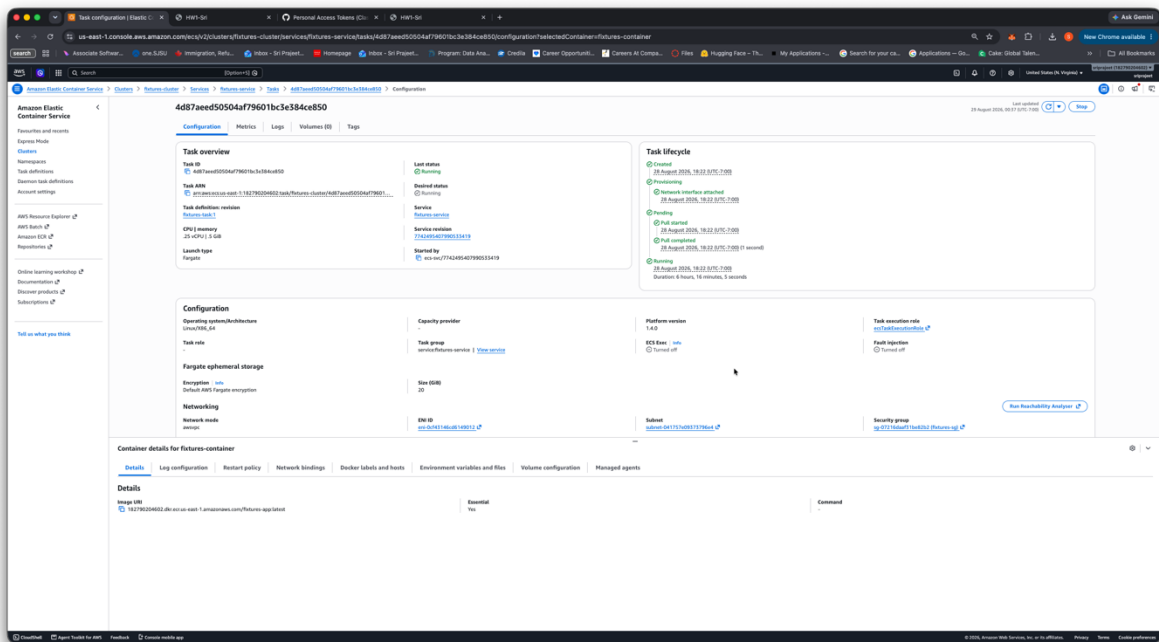
created a security group called fixtures-sg



Created a Fargate cluster called fixtures-cluster



Once the task was running, I opened it and found its public IP in the networking details. Going to that IP in a browser loaded the app, now live on the public internet.



Agentic AI (Part 2)

I built a small pipeline of agents that read a sports fixture and turn it into three topical tags and a short summary. A Planner proposes the tags and summary, a Reviewer checks that work, and a Finalizer produces the clean final answer, and the whole thing prints out as JSON.

Environment setup

Checking Python and conda

```
villyalsriprajeet — -zsh — 80x24

Last login: Fri Aug 28 16:48:28 on ttys004
[(base) villyalsriprajeet@Villyals-MacBook-Pro ~ % python --version
Python 3.13.5
[(base) villyalsriprajeet@Villyals-MacBook-Pro ~ % conda --version
conda 25.7.0
(base) villyalsriprajeet@Villyals-MacBook-Pro ~ %
```

created a separate conda environment named data260 pinned to Python 3.12, activated it, and installed the libraries.

```
villyalsriprajeet — -zsh — 92x30

[(data260) villyalsriprajeet@Villyals-MacBook-Pro ~ % pip install langchain langchain-core langchain-ollama
Collecting langchain
  Downloading langchain-1.3.18-py3-none-any.whl.metadata (6.1 kB)
Collecting langchain-core
  Downloading langchain_core-1.6.1-py3-none-any.whl.metadata (4.8 kB)
Collecting langchain-ollama
  Downloading langchain_ollama-1.1.0-py3-none-any.whl.metadata (3.0 kB)
Collecting langgraph<1.3.0,>=1.2.11 (from langchain)
  Downloading langgraph-1.2.11-py3-none-any.whl.metadata (4.9 kB)
Collecting pydantic<3.0.0,>=2.7.4 (from langchain)
  Downloading pydantic-2.13.5-py3-none-any.whl.metadata (110 kB)
Collecting httpx<1.0.0,>=0.23.0 (from langchain-core)
  Using cached httpx-0.28.1-py3-none-any.whl.metadata (7.1 kB)
Collecting jsonpatch<2.0.0,>=1.33.0 (from langchain-core)
  Downloading jsonpatch-1.33-py2.py3-none-any.whl.metadata (3.0 kB)
Collecting langchain-protocol>=0.0.17 (from langchain-core)
  Downloading langchain_protocol-0.0.19-py3-none-any.whl.metadata (2.4 kB)
Collecting langsmith<1.0.0,>=0.3.45 (from langchain-core)
  Downloading langsmith-0.11.2-py3-none-any.whl.metadata (22 kB)
Requirement already satisfied: packaging>=23.2.0 in ./miniconda3_x86/envs/data260/lib/python3.12/site-packages (from langchain-core) (26.3)
Collecting pyyaml<7.0.0,>=5.3.0 (from langchain-core)
  Downloading pyyaml-6.0.3-cp312-cp312-macosx_10_13_x86_64.whl.metadata (2.4 kB)
Collecting tenacity!=8.4.0,<10.0.0,>=8.1.0 (from langchain-core)
  Downloading tenacity-9.1.4-py3-none-any.whl.metadata (1.2 kB)
Collecting typing-extensions<5.0.0,>=4.7.0 (from langchain-core)
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)
Collecting uuid-utils<1.0,>=0.12.0 (from langchain-core)
  Downloading uuid_utils-0.17.0-cp312-cp312-macosx_10_12_x86_64.whl.metadata (6.4 kB)
```

```
villyalsriprajeet — zsh — 92x30

Downloading langchain_protocol-0.0.19-py3-none-any.whl (7.3 kB)
Downloading orjson-3.12.0-cp312-cp312-macosx_10_15_x86_64.macosx_11_0_arm64.macosx_10_15_uni
versal2.whl (223 kB)
Downloading ormsgpack-1.12.2-cp312-cp312-macosx_10_12_x86_64.macosx_11_0_arm64.macosx_10_12_
universal2.whl (378 kB)
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.1-cp312-cp312-macosx_10_13_universal2.whl (344 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Downloading requests_toolbelt-1.0.0-py2.py3-none-any.whl (54 kB)
Using cached sniffio-1.3.1-py3-none-any.whl (10 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading xxhash-4.0.1-cp312-cp312-macosx_10_13_x86_64.whl (38 kB)
Downloading zstandard-0.25.0-cp312-cp312-macosx_10_13_x86_64.whl (795 kB)
795.7/795.7 kB 3.9 MB/s 0:00:00
Installing collected packages: zstandard, xxhash, websockets, uuid-utils, urllib3, typing-ex
tensions, tenacity, sniffio, pyyaml, ormsgpack, orjson, jsonpointer, idna, h11, distro, char
set_normalizer, certifi, annotated-types, typing-inspection, requests, pydantic-core, langch
ain-protocol, jsonpatch, httpcore, anyio, requests-toolbelt, pydantic, httpx, ollama, langsm
ith, langchain-core, langgraph-sdk, langgraph-checkpoint, langchain-ollama, langgraph-prebui
lt, langgraph, langchain
Successfully installed annotated-types-0.8.0 anyio-4.14.2 certifi-2026.7.22 charset_normaliz
er-3.5.1 distro-1.9.0 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.19 jsonpatch-1.33 jsonpo
inter-3.1.1 langchain-1.3.18 langchain-core-1.6.1 langchain-ollama-1.1.0 langchain-protocol-
0.0.19 langgraph-1.2.11 langgraph-checkpoint-4.2.0 langgraph-prebuilt-1.1.0 langgraph-sdk-0.
4.4 langsmith-0.11.2 ollama-0.6.2 orjson-3.12.0 ormsgpack-1.12.2 pydantic-2.13.5 pydantic-co
re-2.46.5 pyyaml-6.0.3 requests-2.34.2 requests-toolbelt-1.0.0 sniffio-1.3.1 tenacity-9.1.4
typing-extensions-4.16.0 typing-inspection-0.4.4 urllib3-2.7.0 uuid-utils-0.17.0 websockets-
16.1.1 xxhash-4.0.1 zstandard-0.25.0
(data260) villyalsriprajeet@Villyals-MacBook-Pro ~ %
```

To make sure the libraries actually imported I ran a quick check that imported everything and printed the langchain version.

```
villyalsriprajeet — zsh — 92x30

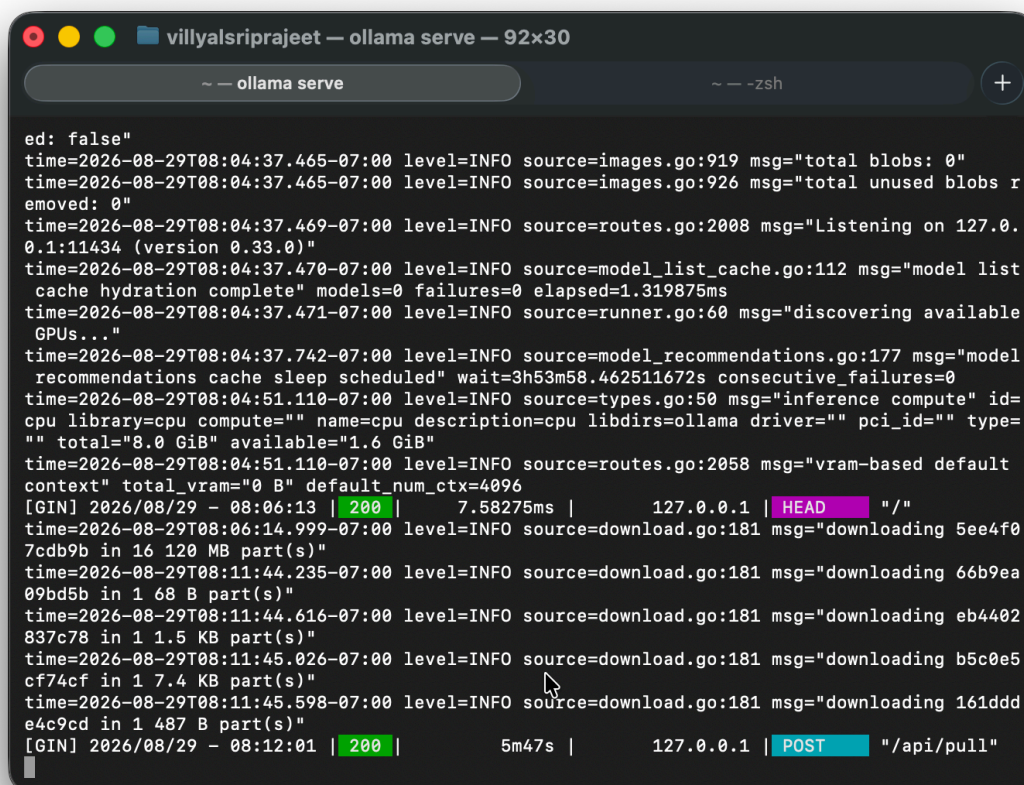
Downloading requests-2.34.2-py3-none-any.whl (73 kB)
Downloading charset_normalizer-3.5.1-cp312-cp312-macosx_10_13_universal2.whl (344 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Downloading certifi-2026.7.22-py3-none-any.whl (136 kB)
Downloading requests_toolbelt-1.0.0-py2.py3-none-any.whl (54 kB)
Using cached sniffio-1.3.1-py3-none-any.whl (10 kB)
Downloading typing_inspection-0.4.4-py3-none-any.whl (14 kB)
Downloading xxhash-4.0.1-cp312-cp312-macosx_10_13_x86_64.whl (38 kB)
Downloading zstandard-0.25.0-cp312-cp312-macosx_10_13_x86_64.whl (795 kB)
795.7/795.7 kB 3.9 MB/s 0:00:00
Installing collected packages: zstandard, xxhash, websockets, uuid-utils, urllib3, typing-ex
tensions, tenacity, sniffio, pyyaml, ormsgpack, orjson, jsonpointer, idna, h11, distro, char
set_normalizer, certifi, annotated-types, typing-inspection, requests, pydantic-core, langch
ain-protocol, jsonpatch, httpcore, anyio, requests-toolbelt, pydantic, httpx, ollama, langsm
ith, langchain-core, langgraph-sdk, langgraph-checkpoint, langchain-ollama, langgraph-prebui
lt, langgraph, langchain
Successfully installed annotated-types-0.8.0 anyio-4.14.2 certifi-2026.7.22 charset_normaliz
er-3.5.1 distro-1.9.0 h11-0.16.0 httpcore-1.0.9 httpx-0.28.1 idna-3.19 jsonpatch-1.33 jsonpo
inter-3.1.1 langchain-1.3.18 langchain-core-1.6.1 langchain-ollama-1.1.0 langchain-protocol-
0.0.19 langgraph-1.2.11 langgraph-checkpoint-4.2.0 langgraph-prebuilt-1.1.0 langgraph-sdk-0.
4.4 langsmith-0.11.2 ollama-0.6.2 orjson-3.12.0 ormsgpack-1.12.2 pydantic-2.13.5 pydantic-co
re-2.46.5 pyyaml-6.0.3 requests-2.34.2 requests-toolbelt-1.0.0 sniffio-1.3.1 tenacity-9.1.4
typing-extensions-4.16.0 typing-inspection-0.4.4 urllib3-2.7.0 uuid-utils-0.17.0 websockets-
16.1.1 xxhash-4.0.1 zstandard-0.25.0
(data260) villyalsriprajeet@Villyals-MacBook-Pro ~ % python -c "import langchain, langchain_
core, langchain_ollama; from langchain_ollama import ChatOllama; print('langchain', langchai
n.__version__); print('imports OK')"
langchain 1.3.18
imports OK
(data260) villyalsriprajeet@Villyals-MacBook-Pro ~ %
```

I then installed ollama through Homebrew and started the local server with ollama serve.

```
[(data260) villyalsriprajeet@Villyals-MacBook-Pro ~ % brew --version
Homebrew 5.1.6
[(data260) villyalsriprajeet@Villyals-MacBook-Pro ~ % brew install ollama
==> Auto-updating Homebrew...
Adjust how often this is run with `HOMEBREW_AUTO_UPDATE_SECS` or disable with
`HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man bre
w`).
==> Downloading https://ghcr.io/v2/homebrew/core/portable-ruby/blobs/sha256:ef0bf45e34c07a11
1674b976e47da3f5e9d5be52eae127b714898ca6949a3c18
##### 100.0%
==> Pouring portable-ruby-4.0.6.catalina.bottle.tar.gz
==> Downloading Homebrew API data
✓ JSON API packages.tahoe.json Downloaded 15.8MB/ 15.8MB
==> Homebrew collects anonymous analytics.
Read the analytics documentation (and how to opt-out) here:
https://docs.brew.sh/Analytics
No analytics have been recorded yet (nor will be during this `brew` run).

==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
https://github.com/Homebrew/brew#-donations
```

I pulled and used qwen2.5:3b instead of qwen3:8b

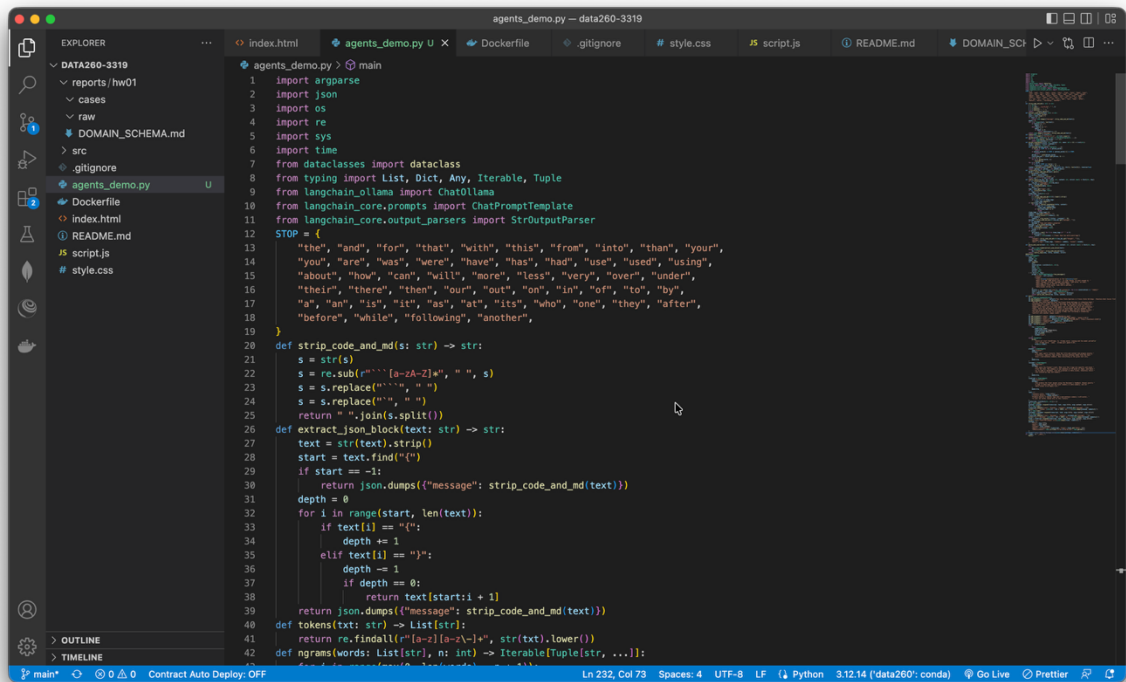


```
villyalsriprajeet — ollama serve — 92x30
~ — ollama serve ~ — zsh +

ed: false"
time=2026-08-29T08:04:37.465-07:00 level=INFO source=images.go:919 msg="total blobs: 0"
time=2026-08-29T08:04:37.465-07:00 level=INFO source=images.go:926 msg="total unused blobs r
emoved: 0"
time=2026-08-29T08:04:37.469-07:00 level=INFO source=routes.go:2008 msg="Listening on 127.0.
0.1:11434 (version 0.33.0)"
time=2026-08-29T08:04:37.470-07:00 level=INFO source=model_list_cache.go:112 msg="model list
cache hydration complete" models=0 failures=0 elapsed=1.319875ms
time=2026-08-29T08:04:37.471-07:00 level=INFO source=runner.go:60 msg="discovering available
GPUs..."
time=2026-08-29T08:04:37.742-07:00 level=INFO source=model_recommendations.go:177 msg="model
recommendations cache sleep scheduled" wait=3h53m58.462511672s consecutive_failures=0
time=2026-08-29T08:04:51.110-07:00 level=INFO source=types.go:50 msg="inference compute" id=
cpu library=cpu compute="" name=cpu description=cpu libdirs=ollama driver="" pci_id="" type=
"" total="8.0 GiB" available="1.6 GiB"
time=2026-08-29T08:04:51.110-07:00 level=INFO source=routes.go:2058 msg="vram-based default
context" total_vram="0 B" default_num_ctx=4096
[GIN] 2026/08/29 - 08:06:13 | 200 | 7.58275ms | 127.0.0.1 | HEAD | "/"
time=2026-08-29T08:06:14.999-07:00 level=INFO source=download.go:181 msg="downloading 5ee4f0
7cdb9b in 16 120 MB part(s)"
time=2026-08-29T08:11:44.235-07:00 level=INFO source=download.go:181 msg="downloading 66b9ea
09bd5b in 1 68 B part(s)"
time=2026-08-29T08:11:44.616-07:00 level=INFO source=download.go:181 msg="downloading eb4402
837c78 in 1 1.5 KB part(s)"
time=2026-08-29T08:11:45.026-07:00 level=INFO source=download.go:181 msg="downloading b5c0e5
cf74cf in 1 7.4 KB part(s)"
time=2026-08-29T08:11:45.598-07:00 level=INFO source=download.go:181 msg="downloading 161ddd
e4c9cd in 1 487 B part(s)"
[GIN] 2026/08/29 - 08:12:01 | 200 | 5m47s | 127.0.0.1 | POST | "/api/pull"
```

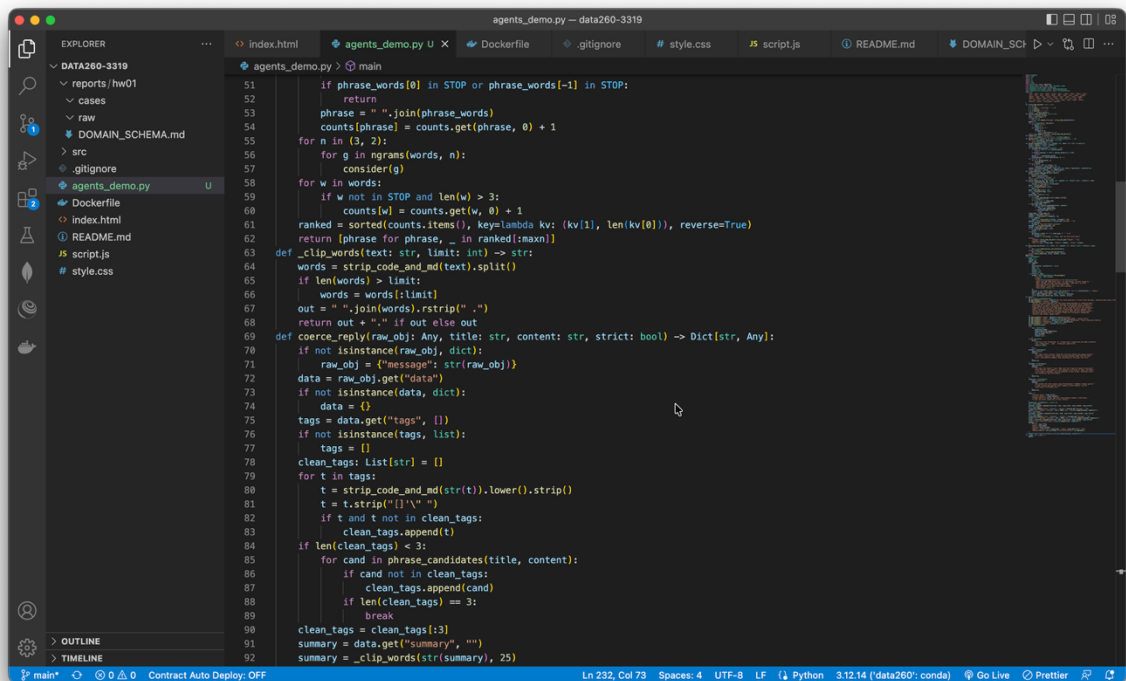
agents_demo.py

created agents_demo.py at the root of my repository. I used a real San Jose State men's soccer fixture.



```
agents_demo.py — data260-3319
EXPLORER
  DATA260-3319
    reports/hw01
    cases
    raw
    DOMAIN_SCHEMA.md
    src
    .gitignore
    agents_demo.py
    Dockerfile
    index.html
    README.md
    script.js
    style.css
  OUTLINE
  TIMELINE
  main*
  Contract Auto Deploy: OFF
Ln 232, Col 79 Spaces: 4 UTF-8 LF Python 3.12.14 (data260: conda) Go Live Prettier

agents_demo.py > main
1 import argparse
2 import json
3 import os
4 import re
5 import sys
6 import time
7 from dataclasses import dataclass
8 from typing import List, Dict, Any, Iterable, Tuple
9 from langchain.globals import ChatOllama
10 from langchain_core.prompts import ChatPromptTemplate
11 from langchain_core.output_parsers import StrOutputParser
12 STOP = {
13     "the", "and", "for", "that", "with", "this", "from", "into", "than", "your",
14     "you", "are", "was", "were", "has", "had", "use", "used", "using",
15     "about", "how", "can", "will", "more", "less", "very", "over", "under",
16     "their", "there", "then", "out", "on", "in", "off", "to", "by",
17     "a", "an", "is", "it", "as", "at", "its", "who", "one", "they", "after",
18     "before", "while", "following", "another",
19 }
20 def strip_code_and_md(s: str) -> str:
21     s = str(s)
22     s = re.sub(r"```[a-zA-Z]*", "", s)
23     s = s.replace("```", "")
24     s = s.replace("\n", " ")
25     return " ".join(s.split())
26 def extract_json_block(text: str) -> str:
27     text = strip_code_and_md(text)
28     start = text.find("{")
29     if start == -1:
30         return json.dumps({"message": strip_code_and_md(text)})
31     depth = 0
32     for i in range(start, len(text)):
33         if text[i] == "{":
34             depth += 1
35         elif text[i] == "}":
36             depth -= 1
37         if depth == 0:
38             return text[start:i + 1]
39     return json.dumps({"message": strip_code_and_md(text)})
40 def tokens(text: str) -> List[str]:
41     return re.findall(r"[a-z1|a-zv|a-z]+", strip_code_and_md(text).lower())
42 def ngrams(words: List[str], n: int) -> Iterable[Tuple[str, ...]]:
```



```
agents_demo.py — data260-3319
EXPLORER
  DATA260-3319
    reports/hw01
    cases
    raw
    DOMAIN_SCHEMA.md
    src
    .gitignore
    agents_demo.py
    Dockerfile
    index.html
    README.md
    script.js
    style.css
  OUTLINE
  TIMELINE
  main*
  Contract Auto Deploy: OFF
Ln 232, Col 73 Spaces: 4 UTF-8 LF Python 3.12.14 (data260: conda) Go Live Prettier

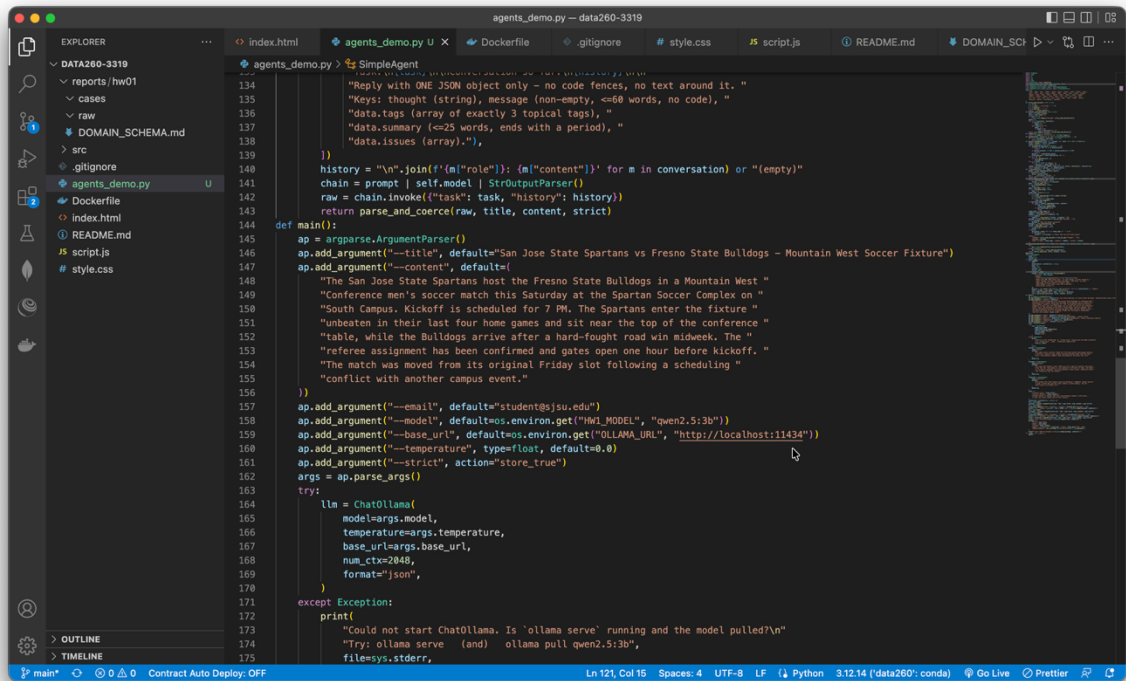
51 if phrase_words[0] in STOP or phrase_words[-1] in STOP:
52     return
53 phrase = " ".join(phrase_words)
54 counts[phrase] = counts.get(phrase, 0) + 1
55 for n in (3, 2):
56     for g in ngrams(words, n):
57         consider(g)
58 for w in words:
59     if w not in STOP and len(w) > 3:
60         counts[w] = counts.get(w, 0) + 1
61 ranked = sorted(counts.items(), key=lambda kv: (kv[1], len(kv[0])), reverse=True)
62 return [phrase for phrase, _ in ranked[:maxn]]
63 def _clip_words(text: str, limit: int) -> str:
64     words = strip_code_and_md(text).split()
65     if len(words) > limit:
66         words = words[:limit]
67     out = " ".join(words.rstrip("."))
68     return out + "." if out else out
69 def coerce_reply(raw_obj: Any, title: str, content: str, strict: bool) -> Dict[str, Any]:
70     if not isinstance(raw_obj, dict):
71         raw_obj = {"message": str(raw_obj)}
72     data = raw_obj.get("data")
73     if not isinstance(data, dict):
74         data = {}
75     tags = data.get("tags", [])
76     if not isinstance(tags, list):
77         tags = []
78     clean_tags: List[str] = []
79     for t in tags:
80         t = strip_code_and_md(str(t)).lower().strip()
81         t = t.strip("[ ]")
82         if t and t not in clean_tags:
83             clean_tags.append(t)
84     if len(clean_tags) < 3:
85         for cand in phrase_candidates(title, content):
86             if cand not in clean_tags:
87                 clean_tags.append(cand)
88             if len(clean_tags) == 3:
89                 break
90     clean_tags = clean_tags[:3]
91     summary = data.get("summary", "")
92     summary = _clip_words(str(summary), 25)
```

```
agents_demo.py — data260-3319
EXPLORER
  DATA260-3319
    reports/hw01
    cases
    raw
    DOMAIN_SCHEMA.md
    src
    .gitignore
    agents_demo.py
    Dockerfile
    index.html
    README.md
    script.js
    style.css
  OUTLINE
  TIMELINE

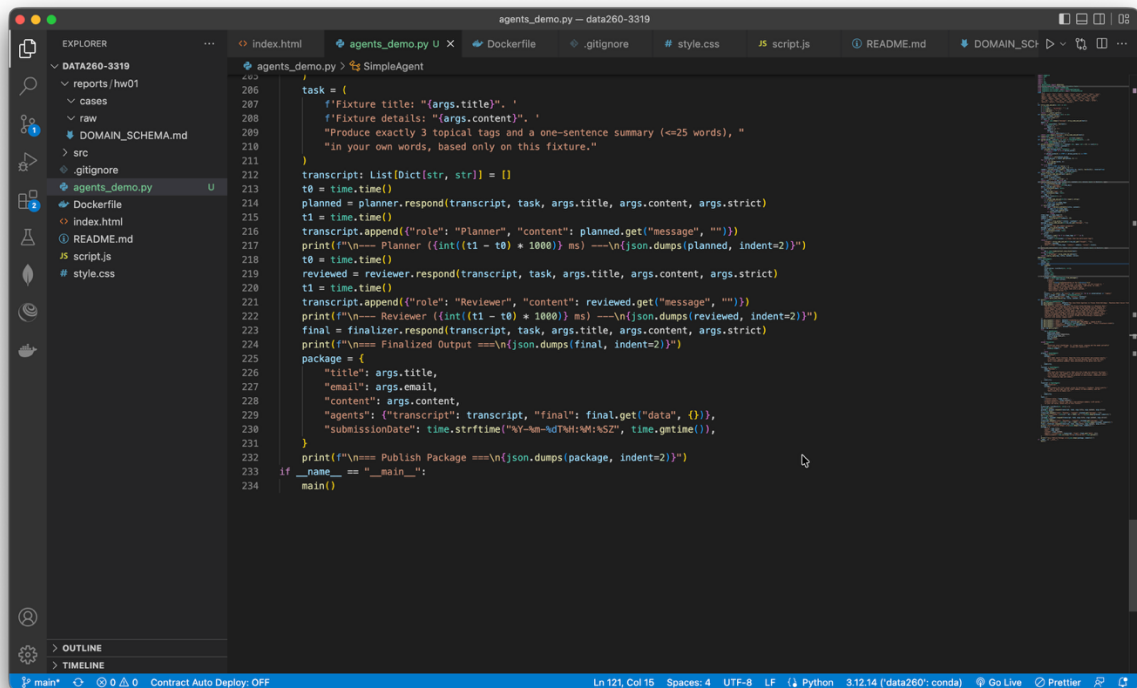
agents_demo.py > SimpleAgent
93 if not summary:
94     summary = _clip_words(f'{title}. {content}', 25)
95     message = strip_code_and_md(str(raw_obj.get("message", "")))
96     if not message:
97         message = "Tags and summary prepared."
98     message = _clip_words(message, 60)
99     issues = data.get("issues", [])
100     if not isinstance(issues, list):
101         issues = []
102     if strict:
103         multiword = sum(1 for t in clean_tags if " " in t)
104         if multiword < 2:
105             issues = list(issues) + ["fewer than two multi-word tags"]
106     return {
107         "thought": strip_code_and_md(str(raw_obj.get("thought", ""))),
108         "message": message,
109         "data": {"tags": clean_tags, "summary": summary, "issues": issues},
110     }
111
112 def parse_and_coerce(text: str, title: str, content: str, strict: bool) -> Dict[str, Any]:
113     try:
114         obj = json.loads(extract_json_block(text))
115     except Exception:
116         obj = {"message": strip_code_and_md(text)}
117     return coerce_reply(obj, title, content, strict)
118
119 @dataclass
120 class SimpleAgent:
121     name: str
122     system: str
123     model: Any
124
125     def respond(
126         self,
127         conversation: List[Dict[str, str]],
128         task: str,
129         title: str,
130         content: str,
131         strict: bool,
132     ) -> Dict[str, Any]:
133         prompt = ChatPromptTemplate.from_messages([
134             ("system", self.system),
135             ("human",
136              f"Task:\n{task}\n\nConversation so far:\n{history}\n\n"
137              "Reply with ONE JSON object only - no code fences, no text around it. ")
138         ])
139         response = self.model.invoke(prompt)
140         return parse_and_coerce(response.get("text", ""), title, content, strict)
141
142 Ln 121, Col 15 Spaces: 4 UTF-8 LF Python 3.12.14 (data260: conda) Go Live Prettier
```

```
agents_demo.py — data260-3319
EXPLORER
  DATA260-3319
    reports/hw01
    cases
    raw
    DOMAIN_SCHEMA.md
    src
    .gitignore
    agents_demo.py
    Dockerfile
    index.html
    README.md
    script.js
    style.css
  OUTLINE
  TIMELINE

agents_demo.py > SimpleAgent
175 if __name__ == "__main__":
176     parser = argparse.ArgumentParser()
177     parser.add_argument("--title", type=str, required=True)
178     parser.add_argument("--content", type=str, required=True)
179     parser.add_argument("--strict", type=bool, default=False)
180     args = parser.parse_args()
181     title = args.title
182     content = args.content
183     strict = args.strict
184
185     planner = SimpleAgent(
186         name="Planner",
187         system=(
188             "You label sports fixtures. Read the title and content and propose exactly "
189             "3 distinct, specific tags (prefer two-word phrases taken from the text) "
190             "plus a one-sentence summary. Base everything on the given text only."
191         ),
192         model=llm,
193     )
194     reviewer = SimpleAgent(
195         name="Reviewer",
196         system=(
197             "You check the Planner's work. Make sure the 3 tags are specific and drawn "
198             "from the text, the summary is one sentence of 25 words or fewer, and there "
199             "is no code or markdown. List any problems in data.issues, otherwise return "
200             "the cleaned-up tags and summary."
201         ),
202         model=llm,
203     )
204     finalizer = SimpleAgent(
205         name="Finalizer",
206         system=(
207             "You produce the final answer using the Reviewer's feedback. Output exactly "
208             "3 tags in data.tags and the final summary in data.summary, and set "
209             "data.issues to an empty list."
210         ),
211         model=llm,
212     )
213
214     task = {
215         f"Fixture title: '{args.title}'. ",
216         f"Fixture details: '{args.content}'. ",
217         "Produce exactly 3 topical tags and a one-sentence summary (<=25 words), "
218         "in your own words, based only on this fixture."
219     }
220
221     transcript: List[Dict[str, str]] = []
222     t0 = time.time()
223     planned = planner.respond(transcript, task, args.title, args.content, args.strict)
224     t1 = time.time()
225     transcript.append({"role": "Planner", "content": planned.get("message", "")})
226
227 Ln 121, Col 15 Spaces: 4 UTF-8 LF Python 3.12.14 (data260: conda) Go Live Prettier
```



```
134 class SimpleAgent:
135     """Reply with ONE JSON object only - no code fences, no text around it.
136     "Keys: thought (string), message (non-empty, <=60 words, no code),
137     "data.tags (array of exactly 3 topical tags),
138     "data.summary (<=25 words, ends with a period),
139     "data.issues (array)."""
140
141     history = "\n".join(f'({m["role"]}): {m["content"]}' for m in conversation) or "(empty)"
142     chain = prompt | self.model | StrOutputParser()
143     raw = chain.invoke({"task": task, "history": history})
144     return parse_and_coerce(raw, title, content, strict)
145
146 def main():
147     ap = argparse.ArgumentParser()
148     ap.add_argument("--title", default="San Jose State Spartans vs Fresno State Bulldogs - Mountain West Soccer Fixture")
149     ap.add_argument("--content", default=(
150         "The San Jose State Spartans host the Fresno State Bulldogs in a Mountain West "
151         "Conference men's soccer match this Saturday at the Spartan Soccer Complex on "
152         "South Campus. Kickoff is scheduled for 7 PM. The Spartans enter the fixture "
153         "unbeaten in their last four home games and sit near the top of the conference "
154         "table, while the Bulldogs arrive after a hard-fought road win midweek. The "
155         "referee assignment has been confirmed and gates open one hour before kickoff. "
156         "The match was moved from its original Friday slot following a scheduling "
157         "conflict with another campus event."
158     ))
159     ap.add_argument("--email", default="student@sjsu.edu")
160     ap.add_argument("--model", default=os.environ.get("HWM_MODEL", "qwen2.5:3b"))
161     ap.add_argument("--base_url", default=os.environ.get("OLLAMA_URL", "http://localhost:11434"))
162     ap.add_argument("--temperature", type=float, default=0.8)
163     ap.add_argument("--strict", action="store_true")
164     args = ap.parse_args()
165     try:
166         llm = ChatOllama(
167             model=args.model,
168             temperature=args.temperature,
169             base_url=args.base_url,
170             num_ctx=2048,
171             format="json",
172         )
173     except Exception:
174         print(
175             "Could not start ChatOllama. Is 'ollama serve' running and the model pulled?\n"
176             "Try: ollama serve (and) ollama pull qwen2.5:3b",
177             file=sys.stderr,
178         )
```



```
206 task = {
207     "Fixture title": "{args.title}",
208     "Fixture details": "{args.content}",
209     "Produce exactly 3 topical tags and a one-sentence summary (<=25 words), "
210     "in your own words, based only on this fixture."
211 }
212 transcript: List[Dict[str, str]] = []
213 t0 = time.time()
214 planned = planner.respond(transcript, task, args.title, args.content, args.strict)
215 t1 = time.time()
216 transcript.append(("Planner", "content": planned.get("message", "")))
217 print(f"\n--- Planner ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(planned, indent=2)}")
218 t0 = time.time()
219 reviewed = reviewer.respond(transcript, task, args.title, args.content, args.strict)
220 t1 = time.time()
221 transcript.append(("Reviewer", "content": reviewed.get("message", "")))
222 print(f"\n--- Reviewer ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(reviewed, indent=2)}")
223 final = finalizer.respond(transcript, task, args.title, args.content, args.strict)
224 print(f"\n--- Finalized Output ---\n{json.dumps(final, indent=2)}")
225 package = {
226     "title": args.title,
227     "email": args.email,
228     "content": args.content,
229     "agents": {"transcript": transcript, "final": final.get("data", {})},
230     "submissionDate": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
231 }
232 print(f"\n--- Publish Package ---\n{json.dumps(package, indent=2)}")
233 if __name__ == "__main__":
234     main()
```

I ran the script with “python agents_demo.py” command. All three agents ran and the pipeline finished cleanly.

```
conda activate data260
(base) villalysr@prajeet@Villalys-MacBook-Pro: data260-3319 % conda activate data260
(data260) villalysr@prajeet@Villalys-MacBook-Pro: data260-3319 % conda activate data260
(data260) villalysr@prajeet@Villalys-MacBook-Pro: data260-3319 % python agents_demo.py

--- Planner (139463 ms) ---
{
  "thought": "I need to extract key information from the text to create tags and a summary.",
  "message": "I'll focus on teams, location, and competition details to create the tags and summary.",
  "data": {
    "tags": [
      "spartans vs bulldogs",
      "mountain west soccer",
      "san jose state vs fresno state"
    ],
    "summary": "San Jose State Spartans host Fresno State Bulldogs in a Mountain West soccer match.",
    "issues": []
  }
}

--- Reviewer (173179 ms) ---
{
  "thought": "I'll focus on teams, location, and competition details to create the tags and summary.",
  "message": "I'll use 'San Jose State Spartans', 'Fresno State Bulldogs', 'Spartan Soccer Complex', and 'Mountain West Conference' as tags. Summary: 'San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.'.",
  "data": {
    "tags": [
      "san jose state spartans",
      "fresno state bulldogs",
      "spartan soccer complex"
    ],
    "summary": "San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.",
    "issues": []
  }
}

=== Finalized Output ===
{
  "thought": "I'll use the provided team names, location, and competition details to create the tags and summary.",
  "message": "I'll use 'San Jose State Spartans', 'Fresno State Bulldogs', 'Spartan Soccer Complex', and 'Mountain West Conference' as tags. Summary: 'San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.'.",
  "data": {
    "tags": [
      "san jose state spartans",
      "fresno state bulldogs",
      "spartan soccer complex"
    ],
    "summary": "San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.",
    "issues": []
  }
}

=== Publish Package ===
{
  "title": "San Jose State Spartans vs Fresno State Bulldogs - Mountain West Soccer Fixture",
  "email": "student@su.edu",
  "content": "The San Jose State Spartans host the Fresno State Bulldogs in a Mountain West Conference men's soccer match this Saturday at the Spartan Soccer Complex on South Campus. Kickoff is scheduled for 7:42 PM. The Spartans enter the fixture unbeaten in their last four home games and sit near the top of the conference table, while the Bulldogs arrive after a hard-fought road win midweek. The referee assignment has been confirmed and gates open one hour before kickoff. The match was moved from its original Friday slot following a scheduling conflict with another campus event.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "I'll focus on teams, location, and competition details to create the tags and summary."
      },
      {
        "role": "Reviewer",
        "content": "I'll use 'San Jose State Spartans', 'Fresno State Bulldogs', 'Spartan Soccer Complex', and 'Mountain West Conference' as tags. Summary: 'San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.'."
      }
    ],
    "final": {
      "tags": [
        "san jose state spartans",
        "fresno state bulldogs",
        "spartan soccer complex"
      ],
      "summary": "San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.",
      "issues": []
    },
    "submissionDate": "2026-08-29T16:47:44Z"
  }
}
```

```
=== Publish Package ===
{
  "title": "San Jose State Spartans vs Fresno State Bulldogs - Mountain West Soccer Fixture",
  "email": "student@su.edu",
  "content": "The San Jose State Spartans host the Fresno State Bulldogs in a Mountain West Conference men's soccer match this Saturday at the Spartan Soccer Complex on South Campus. Kickoff is scheduled for 7:42 PM. The Spartans enter the fixture unbeaten in their last four home games and sit near the top of the conference table, while the Bulldogs arrive after a hard-fought road win midweek. The referee assignment has been confirmed and gates open one hour before kickoff. The match was moved from its original Friday slot following a scheduling conflict with another campus event.",
  "agents": {
    "transcript": [
      {
        "role": "Planner",
        "content": "I'll focus on teams, location, and competition details to create the tags and summary."
      },
      {
        "role": "Reviewer",
        "content": "I'll use 'San Jose State Spartans', 'Fresno State Bulldogs', 'Spartan Soccer Complex', and 'Mountain West Conference' as tags. Summary: 'San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.'."
      }
    ],
    "final": {
      "tags": [
        "san jose state spartans",
        "fresno state bulldogs",
        "spartan soccer complex"
      ],
      "summary": "San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match.",
      "issues": []
    },
    "submissionDate": "2026-08-29T16:47:44Z"
  }
}
```

The three agents :

Each agent is the same small building block name, a short instruction, and the shared model with different instructions:

Planner: reads the fixture and proposes three specific tags plus a one line summary, based only on the text.

Reviewer: checks the Planner's tags and summary against the rules and either lists problems or passes the cleaned version along.

Finalizer: takes the Reviewer's feedback and produces the final answer exactly three tags, the final summary, and an empty issues list.

They run one after another, and each can see the running transcript of what came before.

The script prints four labelled blocks: Planner, Reviewer, Finalized Output, and a Publish Package that shows the title, email, content, transcript, final data and a submission date.

Q1. The three final tags

san jose state spartans, fresno state bulldogs, spartan soccer complex

Q2. The final summary

"San Jose State Spartans host Fresno State Bulldogs in a Mountain West Conference soccer match."

Q3. Did the Reviewer agent change anything?

Yes. The Planner first proposed the tags "spartans vs bulldogs", "mountain west soccer", and "san jose state vs fresno state". The Reviewer changed them to more specific ones "san jose state spartans", "fresno state bulldogs", and "spartan soccer complex" swapping in the full team names and the venue rather than the shorter matchup phrases, and its message explains that it chose to use those names plus the conference. The Finalizer then kept the Reviewer's set as the final answer. So the Reviewer genuinely refined the Planner's work.

Summary

I set up an isolated Python 3.12 environment and a local Ollama model. The fixture text goes in; the Planner reads it and pitches three tags and a summary; the Reviewer double checks those against the rules and either flags problems or approves them; and the Finalizer produces the final clean JSON.

Part 3: Measuring Non-Determinism

The fixed input is my San Jose State vs Fresno State soccer fixture, saved as reports/hw01/cases/nondeterminism_input.json and left unchanged for the whole experiment. I wrote a runner script (nondeterminism_run.py) that imports the same agents from agents_demo.py used in Part 2, so the experiment runs the real pipeline rather than a copy of it. The runner loops 20 times at each temperature, times each run, records the final tags, and saves everything to reports/hw01/raw/ as all_runs.json and all_runs.csv, with the computed metrics in metrics.json.

Because each full run takes several minutes on my hardware, I set **num_ctx to 512** and **num_predict to 256** to keep the per run time manageable.

Metrics Result

```
===== METRICS =====
```

```
Temperature 0.0:
  Distinct tag sets      : 2
  Tags in all 20 runs    : ['spartan soccer complex']
  Tags in exactly 1 run  : ['fresno state bulldogs', 'san jose state spartans']
  Latency p50/p95/p99 ms : 330732.5 / 476172.0 / 479911.2
```

```
Temperature 0.7:
  Distinct tag sets      : 14
  Tags in all 20 runs    : []
  Tags in exactly 1 run  : ['college_venues', 'conference table', 'conferencegames', 'fixture_details', 'fresno bulldogs', 'fresno state bulldogs', 'mountainwestsoccer', 'san jose state home game', 'san jose state spartans', 'scheduling conflict', 'soccer match', 'soccer_teams', 'soccerfixtures', 'spartan soccer', 'spartanmatch']
  Latency p50/p95/p99 ms : 397549.5 / 531446.2 / 532756.4
```

Temperature 0.0:

Distinct tag sets : 2

Tags in all 20 runs : ['spartan soccer complex']

Tags in exactly 1 run : ['fresno state bulldogs', 'san jose state spartans']

Latency p50/p95/p99 ms : 330732.5 / 476172.0 / 479911.2

Temperature 0.7:

Distinct tag sets : 14

Tags in all 20 runs : []

Tags in exactly 1 run : ['college_venues', 'conference table', 'conferencegames', 'fixture_details', 'fresno bulldogs', 'fresno state bulldogs', 'mountainwestsoccer', 'san jose state home game', 'san jose state spartans', 'scheduling conflict', 'soccer match', 'soccer_teams', 'soccerfixtures', 'spartan soccer', 'spartanmatch']

Latency p50/p95/p99 ms : 397549.5 / 531446.2 / 532756.4

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL JUPYTER TRUFFLE
● (data260) villyalsriprajee@Mac data260-3319 % caffeine -i python nondeterminism_run.py | tee reports/hw01/RUN_LOG.txt
Fixed input loaded: San Jose State Spartans vs Fresno State Bulldogs - Mountain West Soccer Fixture

Starting 20 runs x 2 temperatures = 40 total runs.

===== Temperature 0.0 : 20 runs =====
[2026-08-29 15:14:04] run 1/20 475926 ms tags=['san jose state spartans', 'fresno state bulldogs', 'spartan soccer complex']
[2026-08-29 15:22:00] run 2/20 480846 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 15:30:01] run 3/20 426321 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 15:37:09] run 4/20 330193 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 15:42:39] run 5/20 345778 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 15:48:25] run 6/20 311365 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 15:53:36] run 7/20 325877 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 15:59:02] run 8/20 307956 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:04:10] run 9/20 367921 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:10:18] run 10/20 331272 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:15:49] run 11/20 320738 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:21:10] run 12/20 309107 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:26:19] run 13/20 371491 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:32:31] run 14/20 317862 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:37:48] run 15/20 313169 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:43:02] run 16/20 309972 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:48:12] run 17/20 367992 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:54:20] run 18/20 304383 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 16:59:24] run 19/20 423694 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']
[2026-08-29 17:06:30] run 20/20 416459 ms tags=['spartans vs bulldogs', 'mountain west soccer', 'spartan soccer complex']

===== Temperature 0.7 : 20 runs =====
[2026-08-29 17:13:26] run 1/20 332229 ms tags=['spartan vs bulldogs', 'mountain west soccer', 'soccer complex']
[2026-08-29 17:19:00] run 2/20 339290 ms tags=['san jose state spartans', 'fresno state bulldogs', 'mountain west soccer']
[2026-08-29 17:24:40] run 3/20 357693 ms tags=['soccer', 'mountainwestconference', 'spartansoccercomplex']
[2026-08-29 17:30:37] run 4/20 421832 ms tags=['soccer', 'mountainwestconference', 'spartanmatch']
[2026-08-29 17:37:39] run 5/20 416374 ms tags=['san jose state', 'fresno state', 'mountain west soccer']
[2026-08-29 17:44:36] run 6/20 316258 ms tags=['soccerfixtures', 'mountainwestsoccer', 'conferencegames']
[2026-08-29 17:49:52] run 7/20 533084 ms tags=['mountain west soccer', 'spartan soccer complex', 'soccer match']
[2026-08-29 17:58:45] run 8/20 520271 ms tags=['san jose state', 'fresno state', 'mountain west soccer']
[2026-08-29 18:07:25] run 9/20 487046 ms tags=['spartan vs bulldogs', 'mountain west soccer', 'scheduling conflict']
[2026-08-29 18:15:32] run 10/20 531360 ms tags=['state', 'spartans', 'bulldogs']
[2026-08-29 18:24:24] run 11/20 484994 ms tags=['spartan vs bulldogs', 'mountain west soccer', 'san jose state home game']
[2026-08-29 18:32:29] run 12/20 418688 ms tags=['state', 'spartans', 'bulldogs']
[2026-08-29 18:39:27] run 13/20 347321 ms tags=['spartan vs bulldogs', 'mountain west soccer', 'soccer complex']
[2026-08-29 18:45:15] run 14/20 388023 ms tags=['state', 'spartans', 'bulldogs']
[2026-08-29 18:51:43] run 15/20 302493 ms tags=['spartan soccer', 'mountain west soccer', 'fresno bulldogs']
[2026-08-29 18:56:45] run 16/20 376442 ms tags=['state', 'spartans', 'bulldogs']
[2026-08-29 19:03:03] run 17/20 385021 ms tags=['soccerfixtures', 'mountainwestconference', 'spartansoccercomplex']
[2026-08-29 19:09:28] run 18/20 422695 ms tags=['soccer teams', 'college venues', 'fixture_details']
[2026-08-29 19:16:31] run 19/20 328962 ms tags=['state', 'spartans', 'bulldogs']
[2026-08-29 19:22:00] run 20/20 407076 ms tags=['mountain west soccer', 'spartan soccer complex', 'conference table']
```

At temperature 0.0 the results stayed consistent across the 20 runs I only got 2 different tag sets, and "spartan soccer complex" came up every time. At temperature 0.7 it was a lot more mixed: 20 runs gave 14 different tag sets, nothing showed up in all of them, and plenty of tags only appeared once. So turning up the temperature clearly added more variation.

The higher temperature also seemed to affect the quality a bit. A few of the 0.7 runs gave messy tags words split apart like "state", "spartans", "bulldogs", or squished together like "mountainwestsoccer". At 0.0 the tags stayed clean and easy to read. The speed was about the same either way, just slightly slower at 0.7, but the real difference showed up in the tags rather than the timing.

Explain what two users sending identical input might see

If two people sent the exact same fixture to this pipeline, at temperature 0.0 they would almost always get the same tags and summary as each other, because the model is being pushed to pick its single most likely answer each time. At temperature 0.7 the same two people could easily get different tag sets, because the model is sampling more randomly. My own numbers show this directly: 2 distinct outcomes at 0.0 versus 14 at 0.7 for identical input.

Example where run-to-run variation is acceptable and one where it is not.

Whether that variation is a problem depends on what the output is used for:

Acceptable: if the tags are just suggestions to help someone label a fixture on a community sports blog, a little run to run variation is fine. If one run says "soccer match" and another says "conference table", a human can still pick what they like, and the small differences don't cause harm.

Not acceptable: if the tags feed an automated system for example, routing fixtures into database categories, triggering ticket sales, or anything that needs to match exactly every time then getting a different tag set on identical input would break things. Two users submitting the same fixture must land in the same category, so here the variation at temperature 0.7 would be unacceptable and I would use temperature 0.0 or lower and add validation.

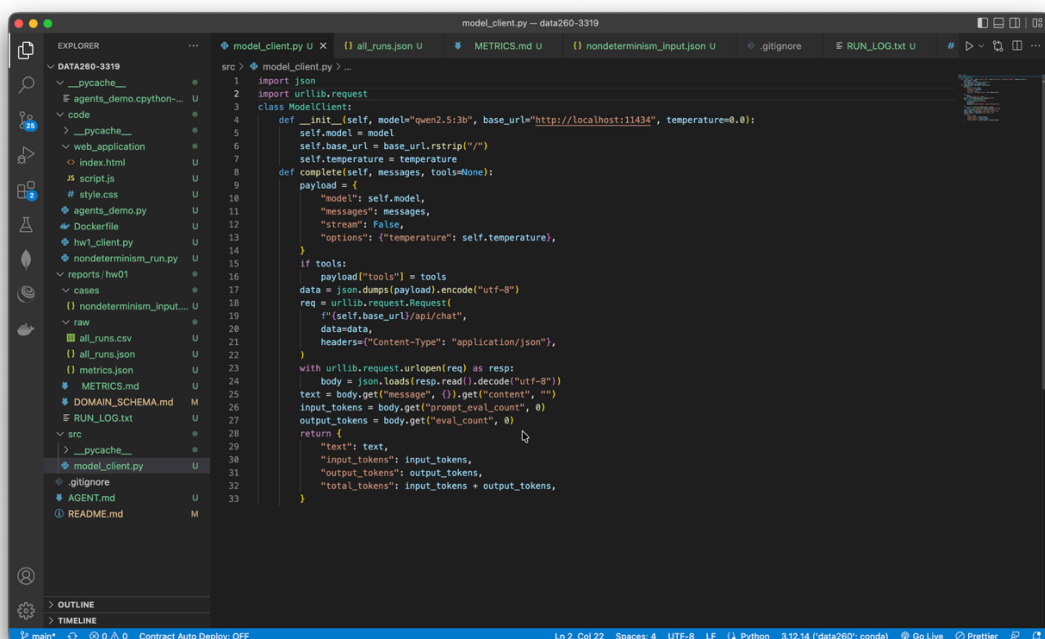
Summary

Running the same input 40 times made the tradeoff clear. Temperature 0.0 keeps things clean and repeatable but with less variety, while 0.7 gives more variety but loses some consistency and quality along the way.

Part 4: Model Client and Token Accounting

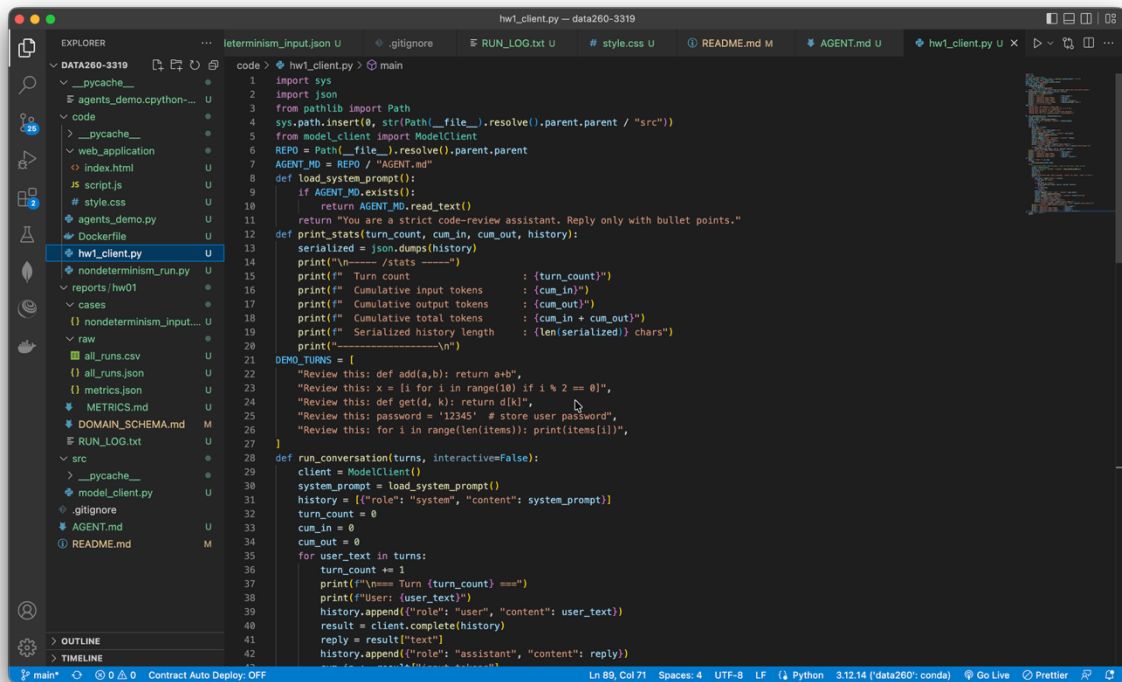
Created a reusable model adapter module at `src/model_client.py`, defining a stable interface such as `complete(messages, tools=None)`. All model calls go through this adapter.

`model_client.py`



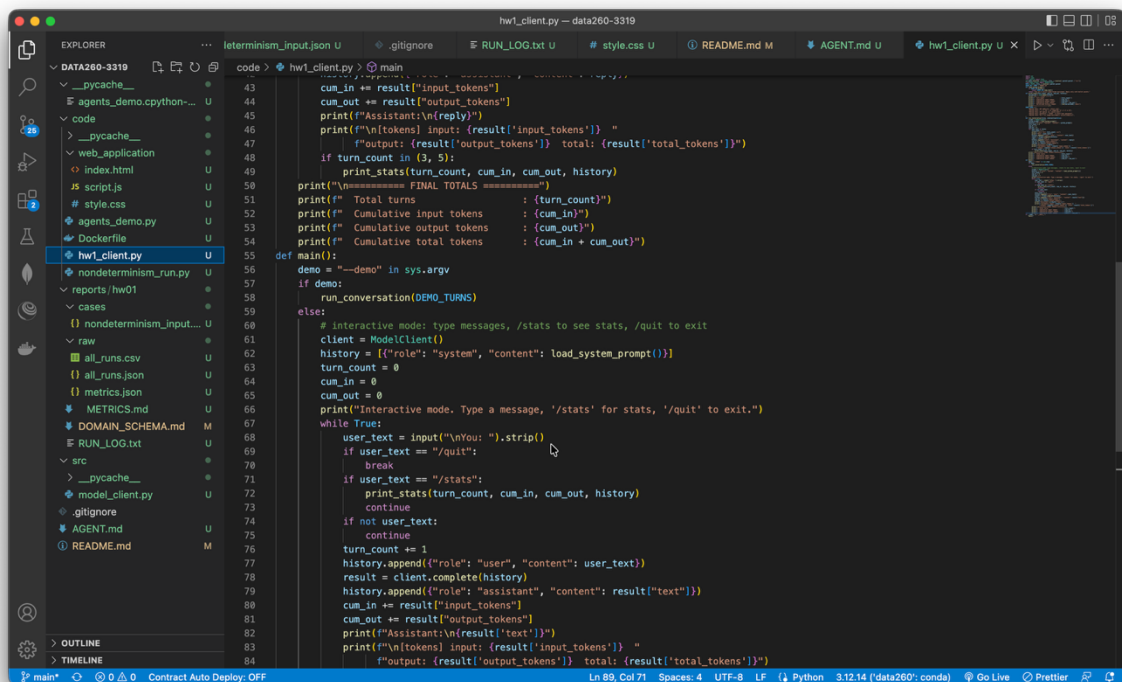
```
model_client.py - data260-3319
src > model_client.py > ...
1 import json
2 import urllib.request
3
4 class ModelClient:
5
6     def __init__(self, model="gpt-3.5-turbo", base_url="http://localhost:11434", temperature=0.0):
7         self.model = model
8         self.base_url = base_url.rstrip("/")
9         self.temperature = temperature
10
11     def complete(self, messages, tools=None):
12         payload = {
13             "model": self.model,
14             "messages": messages,
15             "stream": False,
16             "options": {"temperature": self.temperature},
17         }
18         if tools:
19             payload["tools"] = tools
20         data = json.dumps(payload).encode("utf-8")
21         req = urllib.request.Request(
22             f"{self.base_url}/api/chat",
23             data=data,
24             headers={"Content-Type": "application/json"},
25         )
26         with urllib.request.urlopen(req) as resp:
27             body = json.loads(resp.read().decode("utf-8"))
28             text = body.get("message", {}).get("content", "")
29             input_tokens = body.get("prompt_eval_count", 0)
30             output_tokens = body.get("eval_count", 0)
31             return {
32                 "text": text,
33                 "input_tokens": input_tokens,
34                 "output_tokens": output_tokens,
35                 "total_tokens": input_tokens + output_tokens,
36             }
```

Created hw1_client.py as a small command line demo that imports the adapter.



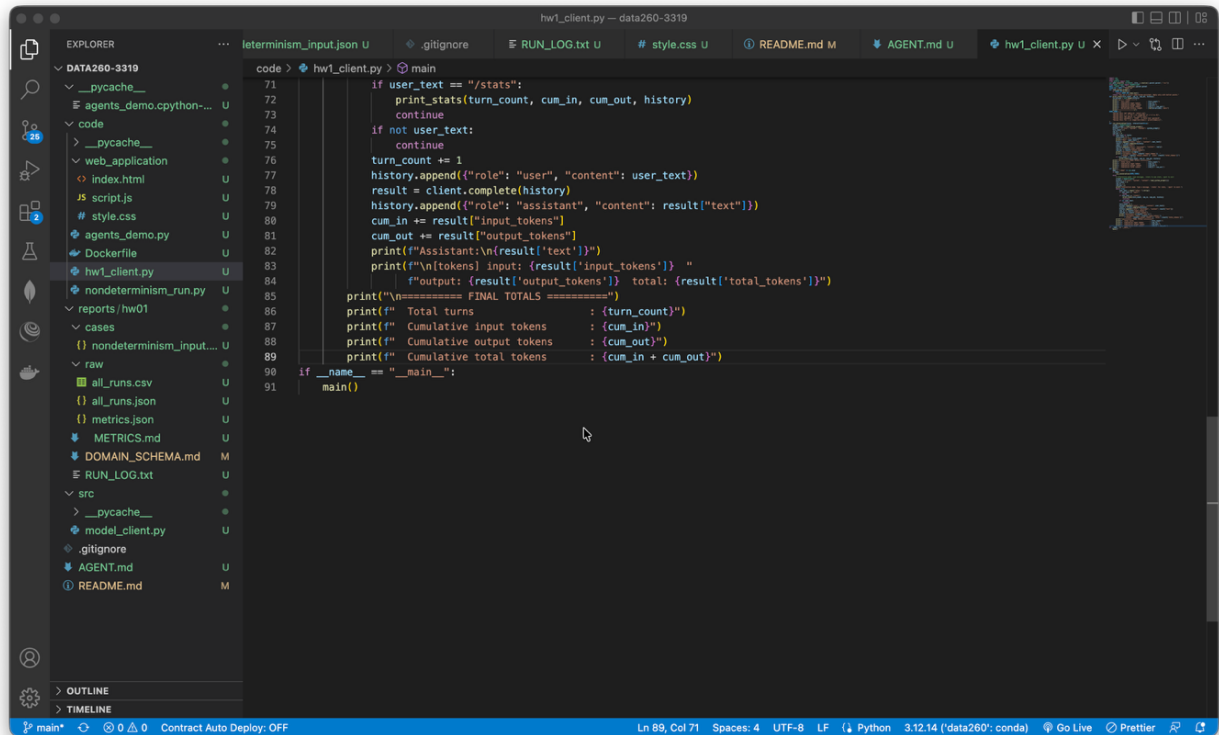
```
hw1_client.py -- data260-3319
EXPLORER  ...  leternism_input.json U  .gitignore  RUN_LOG.txt U  style.css U  README.md M  AGENT.md U  hw1_client.py U
DATA260-3319
  __pycache__
  agents_demo.cpython-...
  code
  __pycache__
  web_application
    index.html U
    script.js U
    style.css U
  agents_demo.py U
  Dockerfile
  hw1_client.py U
  nondeterminism_run.py U
  reports/hw01
    cases
    nondeterminism_input... U
    raw
  all_runs.csv U
  all_runs.json U
  metrics.json U
  METRICS.md U
  DOMAIN_SCHEMA.md M
  RUN_LOG.txt
  src
    __pycache__
    model_client.py U
    .gitignore
    AGENT.md U
    README.md M
  main
  0 0 Contract Auto Deploy: OFF
Ln 89, Col 71 Spaces: 4 UTF-8 LF Python 3.12.14 ('data260': conda) Go Live Prettier
```

```
code > hw1_client.py > main
1 import sys
2 import json
3 from pathlib import Path
4 sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "src"))
5 from model_client import ModelClient
6 REPO = Path(__file__).resolve().parent.parent
7 AGENT_MD = REPO / "AGENT.md"
8 def load_system_prompt():
9     if AGENT_MD.exists():
10         return AGENT_MD.read_text()
11     return "You are a strict code-review assistant. Reply only with bullet points."
12 def print_stats(turn_count, cum_in, cum_out, history):
13     serialized = json.dumps(history)
14     print(f"\n----- /stats -----")
15     print(f" Turn count           : {turn_count}")
16     print(f" Cumulative input tokens : {cum_in}")
17     print(f" Cumulative output tokens : {cum_out}")
18     print(f" Cumulative total tokens : {cum_in + cum_out}")
19     print(f" Serialized history length : {len(serialized)} chars")
20     print(f"-----\n")
21 DEMO_TURNS = [
22     "Review this: def add(a,b): return a+b",
23     "Review this: x = {i for i in range(10) if i % 2 == 0}",
24     "Review this: def get(d, k): return d[k]",
25     "Review this: password = '12345' # store user password",
26     "Review this: for i in range(len(items)): print(items[i])",
27 ]
28 def run_conversation(turns, interactive=False):
29     client = ModelClient()
30     system_prompt = load_system_prompt()
31     history = [{"role": "system", "content": system_prompt}]
32     turn_count = 0
33     cum_in = 0
34     cum_out = 0
35     for user_text in turns:
36         turn_count += 1
37         print(f"\n==== Turn {turn_count} ====")
38         print(f"User: {user_text}")
39         history.append({"role": "user", "content": user_text})
40         result = client.complete(history)
41         reply = result["text"]
42         history.append({"role": "assistant", "content": reply})
43         cum_in += result["input_tokens"]
44         cum_out += result["output_tokens"]
45         print(f"Assistant:\n{reply}")
46         print(f"\n[tokens] input: {result['input_tokens']} "
47               f"output: {result['output_tokens']} total: {result['total_tokens']}")
48         if turn_count in (3, 5):
49             print_stats(turn_count, cum_in, cum_out, history)
50     print(f"\n===== FINAL TOTALS =====")
51     print(f" Total turns           : {turn_count}")
52     print(f" Cumulative input tokens : {cum_in}")
53     print(f" Cumulative output tokens : {cum_out}")
54     print(f" Cumulative total tokens : {cum_in + cum_out}")
55 def main():
56     demo = "--demo" in sys.argv
57     if demo:
58         run_conversation(DEMO_TURNS)
59     else:
60         # Interactive mode: type messages, /stats to see stats, /quit to exit
61         client = ModelClient()
62         history = [{"role": "system", "content": load_system_prompt()}]
63         turn_count = 0
64         cum_in = 0
65         cum_out = 0
66         print("Interactive mode. Type a message, '/stats' for stats, '/quit' to exit.")
67         while True:
68             user_text = input("\nYou: ").strip()
69             if user_text == "/quit":
70                 break
71             if user_text == "/stats":
72                 print_stats(turn_count, cum_in, cum_out, history)
73                 continue
74             if not user_text:
75                 continue
76             turn_count += 1
77             history.append({"role": "user", "content": user_text})
78             result = client.complete(history)
79             history.append({"role": "assistant", "content": result["text"]})
80             cum_in += result["input_tokens"]
81             cum_out += result["output_tokens"]
82             print(f"Assistant:\n{result['text']}")
83             print(f"\n[tokens] input: {result['input_tokens']} "
84                   f"output: {result['output_tokens']} total: {result['total_tokens']}")
```

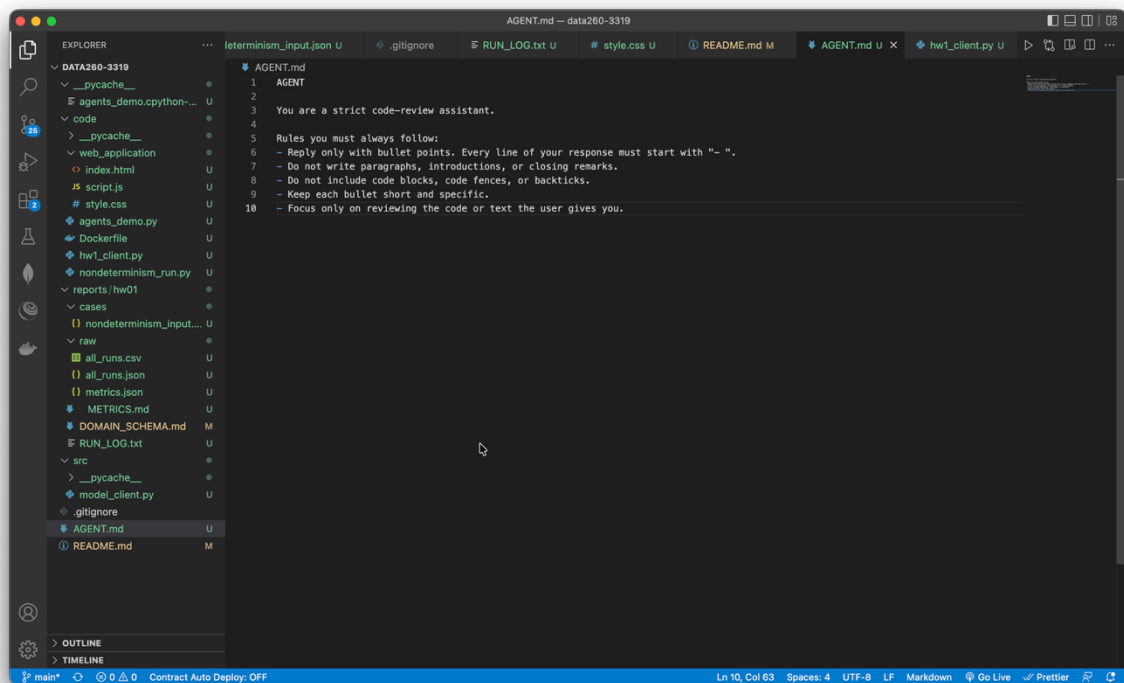


```
hw1_client.py -- data260-3319
EXPLORER  ...  leternism_input.json U  .gitignore  RUN_LOG.txt U  style.css U  README.md M  AGENT.md U  hw1_client.py U
DATA260-3319
  __pycache__
  agents_demo.cpython-...
  code
  __pycache__
  web_application
    index.html U
    script.js U
    style.css U
  agents_demo.py U
  Dockerfile
  hw1_client.py U
  nondeterminism_run.py U
  reports/hw01
    cases
    nondeterminism_input... U
    raw
  all_runs.csv U
  all_runs.json U
  metrics.json U
  METRICS.md U
  DOMAIN_SCHEMA.md M
  RUN_LOG.txt
  src
    __pycache__
    model_client.py U
    .gitignore
    AGENT.md U
    README.md M
  main
  0 0 Contract Auto Deploy: OFF
Ln 89, Col 71 Spaces: 4 UTF-8 LF Python 3.12.14 ('data260': conda) Go Live Prettier
```

```
code > hw1_client.py > main
43 cum_in += result["input_tokens"]
44 cum_out += result["output_tokens"]
45 print(f"Assistant:\n{reply}")
46 print(f"\n[tokens] input: {result['input_tokens']} "
47       f"output: {result['output_tokens']} total: {result['total_tokens']}")
48 if turn_count in (3, 5):
49     print_stats(turn_count, cum_in, cum_out, history)
50 print(f"\n===== FINAL TOTALS =====")
51 print(f" Total turns           : {turn_count}")
52 print(f" Cumulative input tokens : {cum_in}")
53 print(f" Cumulative output tokens : {cum_out}")
54 print(f" Cumulative total tokens : {cum_in + cum_out}")
55 def main():
56     demo = "--demo" in sys.argv
57     if demo:
58         run_conversation(DEMO_TURNS)
59     else:
60         # Interactive mode: type messages, /stats to see stats, /quit to exit
61         client = ModelClient()
62         history = [{"role": "system", "content": load_system_prompt()}]
63         turn_count = 0
64         cum_in = 0
65         cum_out = 0
66         print("Interactive mode. Type a message, '/stats' for stats, '/quit' to exit.")
67         while True:
68             user_text = input("\nYou: ").strip()
69             if user_text == "/quit":
70                 break
71             if user_text == "/stats":
72                 print_stats(turn_count, cum_in, cum_out, history)
73                 continue
74             if not user_text:
75                 continue
76             turn_count += 1
77             history.append({"role": "user", "content": user_text})
78             result = client.complete(history)
79             history.append({"role": "assistant", "content": result["text"]})
80             cum_in += result["input_tokens"]
81             cum_out += result["output_tokens"]
82             print(f"Assistant:\n{result['text']}")
83             print(f"\n[tokens] input: {result['input_tokens']} "
84                   f"output: {result['output_tokens']} total: {result['total_tokens']}")
```



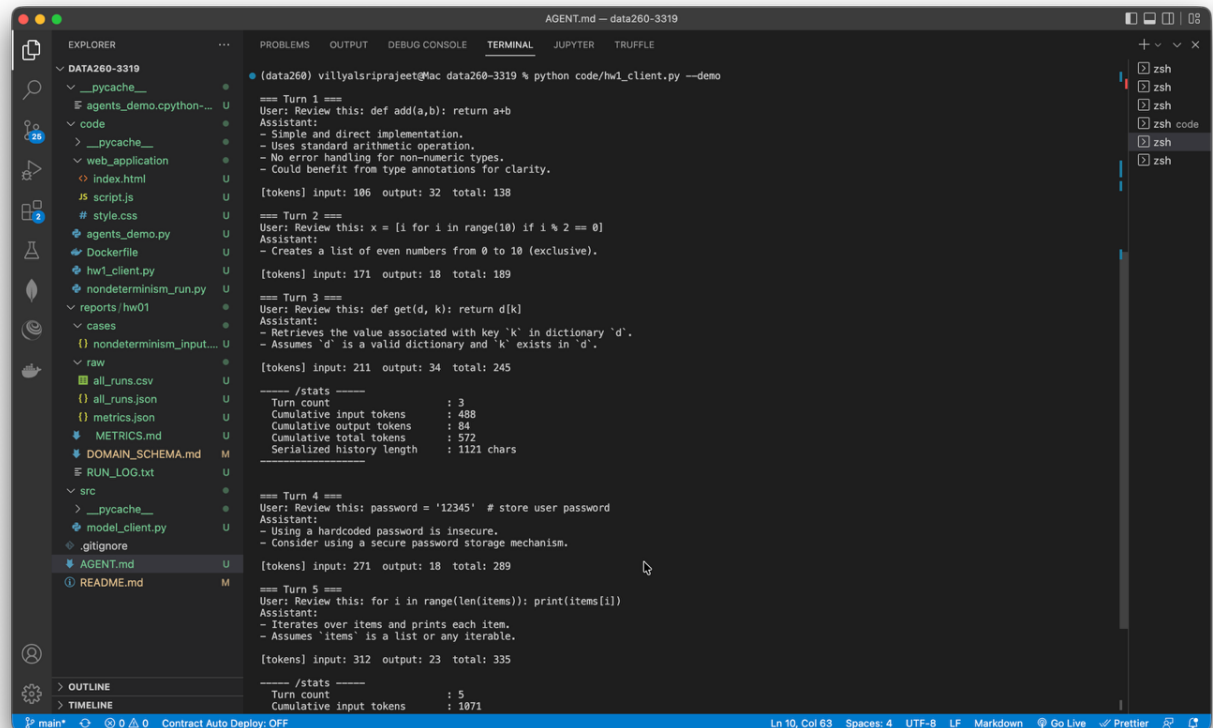
Created AGENT.md asking for strict bullet only code review, and verified the model actually follows it.



I ran the demo from the repo root with python code/hw1_client.py --demo command and got the below results. Made sure after every model response, print input tokens,

output tokens, and total tokens for that turn. On exit, print cumulative input tokens, output tokens, and turn count.

Also added a /stats command showing turn count, cumulative token counts, and serialized conversation-history length, without altering the history.



```
AGENT.md -- data260-3319
(data260) villyalsrprajeet@Mac data260-3319 % python code/hw1_client.py --demo

=== Turn 1 ===
User: Review this: def add(a,b): return a+b
Assistant:
- Simple and direct implementation.
- Uses standard arithmetic operation.
- No error handling for non-numeric types.
- Could benefit from type annotations for clarity.
[tokens] input: 106 output: 32 total: 138

=== Turn 2 ===
User: Review this: x = [1 for i in range(10) if i % 2 == 0]
Assistant:
- Creates a list of even numbers from 0 to 10 (exclusive).
[tokens] input: 171 output: 18 total: 189

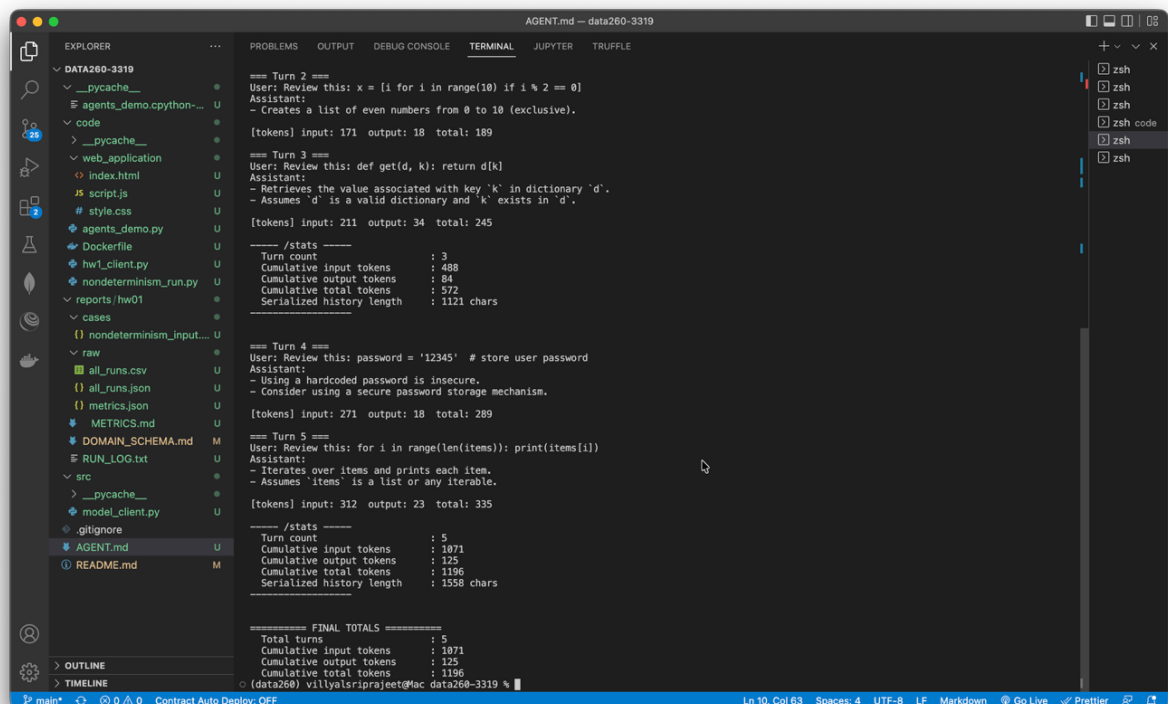
=== Turn 3 ===
User: Review this: def get(d, k): return d[k]
Assistant:
- Retrieves the value associated with key 'k' in dictionary 'd'.
- Assumes 'd' is a valid dictionary and 'k' exists in 'd'.
[tokens] input: 211 output: 34 total: 245

--- /stats ---
Turn count          : 3
Cumulative input tokens : 488
Cumulative output tokens : 84
Cumulative total tokens : 572
Serialized history length : 1121 chars

=== Turn 4 ===
User: Review this: password = '12345' # store user password
Assistant:
- Using a hardcoded password is insecure.
- Consider using a secure password storage mechanism.
[tokens] input: 271 output: 18 total: 289

=== Turn 5 ===
User: Review this: for i in range(len(items)): print(items[i])
Assistant:
- Iterates over items and prints each item.
- Assumes 'items' is a list or any iterable.
[tokens] input: 312 output: 23 total: 335

--- /stats ---
Turn count          : 5
Cumulative input tokens : 1071
```



```
AGENT.md -- data260-3319

=== Turn 2 ===
User: Review this: x = [1 for i in range(10) if i % 2 == 0]
Assistant:
- Creates a list of even numbers from 0 to 10 (exclusive).
[tokens] input: 171 output: 18 total: 189

=== Turn 3 ===
User: Review this: def get(d, k): return d[k]
Assistant:
- Retrieves the value associated with key 'k' in dictionary 'd'.
- Assumes 'd' is a valid dictionary and 'k' exists in 'd'.
[tokens] input: 211 output: 34 total: 245

--- /stats ---
Turn count          : 3
Cumulative input tokens : 488
Cumulative output tokens : 84
Cumulative total tokens : 572
Serialized history length : 1121 chars

=== Turn 4 ===
User: Review this: password = '12345' # store user password
Assistant:
- Using a hardcoded password is insecure.
- Consider using a secure password storage mechanism.
[tokens] input: 271 output: 18 total: 289

=== Turn 5 ===
User: Review this: for i in range(len(items)): print(items[i])
Assistant:
- Iterates over items and prints each item.
- Assumes 'items' is a list or any iterable.
[tokens] input: 312 output: 23 total: 335

--- /stats ---
Turn count          : 5
Cumulative input tokens : 1071
Cumulative output tokens : 125
Cumulative total tokens : 1196
Serialized history length : 1558 chars

===== FINAL TOTALS =====
Total turns          : 5
Cumulative input tokens : 1071
Cumulative output tokens : 125
Cumulative total tokens : 1196
Serialized history length : 1558 chars

(data260) villyalsrprajeet@Mac data260-3319 %
```

Why is prior conversation context resent with every turn?

The model doesn't remember anything between calls, so each time it only sees what I send in that one request. To keep the conversation making sense, I resend the whole history every time that's the only way it knows what was said before.

How is a system prompt different from a user message?

The system prompt sets the rules for the whole conversation and stays at the top the whole time. A user message is just a single request, like "review this code." One shapes how the model behaves overall, the other is what it's replying to that turn.

Why do input tokens grow over a conversation?

Since the full history gets resent each turn, every new turn includes everything before it plus the new message. You can see it in my run the input tokens went 106, 171, 211, 271, 312, growing each time.

What eventually limits that growth?

The model's context window the most tokens it can take in one request. Once the history gets close to that limit, you have to start trimming or summarizing older turns, otherwise it gets cut off. So the context window is the ceiling on how long the conversation can grow.